

ConsistenCy v3

理论基础、数学形式化与研究报告

Theoretical Foundations, Mathematical Formalization
and a Research Agenda

基于 Capability Security、Queueing Theory、Provenance、
Agent Systems 与 Information Freshness 文献的诚实映射

报告类型 理论与技术报告（非产品文档）
研究对象 ConsistenCy v3（分支 v3，工作树状态 2026-08-28）
文献基线 45 篇经网络核验的文献（20 篇承重核心）
版本 v1.0
日期 2026 年 8 月 28 日

本报告不修改任何产品源代码；所有架构事实以当前源码为唯一真相来源。

摘要

本报告对 ConsistenCy v3——一个仓库原生的 Agent Harness 操作系统——进行理论基础研究。以当前工作树（分支 v3，2026-08-28）为唯一事实来源，我们把 14 个经源码核验的组件映射到能力安全、排队与调度、溯源与可复现性、Agent 系统与 workflow 形式方法、信息新鲜度五个理论域，并构造共享记号层面的形式模型 $\mathcal{M}_C$ ：全局状态 X_t （十元组，见附录 C）、九步授权谓词 Auth、撤销支配命题、可用性/可见性与权威的正交性命题、ACB 与 Fiber 状态机的单向耦合关系、workflow DAG 的裁决面独立性、 $M/M/c$ 基线排队模型（显式声明的近似）、证据指纹的确定性与碰撞界、以及版本滞后度量 $D(t)$ 与检测时延界。所有映射按六类支持强度分级（DIRECT、STRONG、USEFUL、PROPOSED、WEAK、NA），全部数学结论按五类标签分类并附假设、证明与失败条件；三组负结果（Petri 网机件、CUSUM 变化检测、线性时间年龄）被显式记录。文献基线为 45 篇经网络核验的文献（20 篇承重核心）；模型经过敌意数学审计（全部 FATAL/MAJOR 修正已并入）与独立引文审计（零虚构）。核心结论：ConsistenCy 的安全内核在**机制层**是引用监视器与对象能力理论的直系实例化（DIRECT；分析性与验证性差距见第 19 章），其“Coeffect $\neq$ Authorization”“Context VM $\neq$ Permission System”公理获得精确数学表述——安全只由 Auth 承担；而调度优化、新鲜度度量与模型检验被诚实标记为提议研究方向，不构成现状声明。

关键词：能力安全；引用监视器；Agent Harness；形式化验证；排队论；数据溯源；信息新鲜度；LLM Agent 治理

目录

第一章 ConsistenCy v3 是什么	1
1.1 一句话定位	1
1.2 产品要回答的五个问题	1
1.3 本报告的证据纪律	2
第二章 为什么 Agent Harness 需要理论基础	3
2.1 问题背景：LLM Agent 的结构性风险	3
2.2 工程直觉为何不够	3
2.3 本报告的立场	4
第三章 当前系统架构与形式化对象	5
3.1 三层物理结构与形式化对象总览	5
3.2 组件清单与工程不变式	6
3.3 权威结构图	7
3.4 形式化对象预览	7
第四章 Capability Security 与 Kernel Authority	9
4.1 理论谱系	9
4.2 授权谓词	10
4.3 安全不变式 S1	11
4.4 撤销支配 (S2)	12
4.5 撤销时间线	12
4.6 与经典理论的差异点	13

第五章 Cordis Lifecycle 与 Authorization 的分离	14
5.1 工程事实	14
5.2 理论映射	14
5.3 可用性与授权的正交性	15
第六章 Context Visibility 与 Authority 的分离	16
6.1 工程事实	16
6.2 可见性与权限的正交性	16
6.3 为什么这不是语义游戏	16
第七章 Agent / Run / Fiber 状态模型	18
7.1 三个状态机，三种所有者	18
7.2 单向耦合积	18
7.3 模型检验扩展 (PROPOSED)	19
第八章 Workflow DAG 与执行语义	20
8.1 四段管线	20
8.2 拓扑序定理与 WF-net 负结果	20
8.3 裁决面独立性 (“校验 $\neq$ 授权”)	21
8.4 执行健全性与活性	22
第九章 排队、调度与资源治理	23
9.1 调度器的真实形状	23
9.2 排队基线模型 (基线近似)	23
9.2.1 假设表 (含 “违背” 列)	25
9.3 LLM 服务时间为何必须非指数建模	26
9.4 CMDP 研究模型 (PROPOSED, 未实现)	26
第十章 仓库监控与事件触发执行	27
10.1 监督子系统的真实形状 (轮询, 而非 Webhook)	27
10.2 事件触发 vs 周期触发: 结构性类比 (STRONG)	27
10.3 CUSUM/变化检测: 不适用 (负结果, N/A)	28

10.4 单运行去重与“缓冲大小为 1”	28
第十一章 证据、溯源与可复现性	29
11.1 证据模型	29
11.2 理论映射	29
11.3 接地不变式（两级严格度，如实）	30
11.4 确定性与指纹可复现性	31
11.5 真相不变式 T1/T2 与可复现性 R1	31
第十二章 仓库与证据新鲜度	33
12.1 为什么线性时间年龄是错误的原始度量（负结果）	33
12.2 版本滞后（PROPOSED 度量，VAoI 接地）	33
12.3 检测时延与新鲜证据时刻（初等界）	34
12.4 平均值与低速率近似（PROPOSED 度量/推导）	34
12.5 AoI 迁移的最终账目	35
第十三章 ConsistenCy 统一数学模型	37
13.1 统一模型（PROPOSED：透镜，非实现）	37
第十四章 形式化安全/活性性质清单	39
第十五章 架构决策的理论解释	41
第十六章 与相关系统/论文的比较	43
第十七章 哪些理论已经支撑当前实现	45
第十八章 哪些只是提议研究方向	47
第十九章 局限	48
19.1 模型与方法论的局限	48
19.2 度量与评估的局限	48
第二十章 未来研究议程	49

附录 A 完整数学推导	51
A.1 Erlang-C 公式的推导梗概（定理 9.2）	51
A.2 Little 定律的证明梗概（定理 9.3）	51
A.3 非抢占 HOL 优先级均值等待（定理 9.5）	52
A.4 碰撞界的完整证明（命题 11.3）	52
A.5 确定性命题（命题 11.2）	52
A.6 新鲜证据时刻上界的完整证明（命题 12.2）	53
A.7 栅格相位均匀性与平均冲刷时延	53
A.8 撤销支配与传播无关性（命题 4.3、5.3）	53
A.9 积 LTS 良定义性（命题 7.2）	54
附录 B 文献映射矩阵	55
附录 C 符号表	58

插图

3.1	架构 $\rightarrow$ 理论域总映射 (Consistent Architecture–Theory Domain Map)。支持强度为六类分级的简写；每条映射的完整论证与关键文献见对应章节及附录 B。	5
3.2	Kernel / Cordis / Agent / Evidence 权威结构图。实线为允许的权威路径；虚线标注被架构排除的路径（触发与描述层数据不能直接成为权威来源，详见命题 8.2）。	8
4.1	能力授权时间线与撤销。上：内核裁决在 t_r 后同步转为拒绝（命题 4.3）；中：Fiber 卸载异步完成，窗口 W_{stale} 内的调用全部被拒；下：撤销前已接受的意图不受影响（carve-out）。	13
6.1	上下文可见性与权威的正交轴。Context VM $\neq$ 权限系统：两轴独立变化，安全性只由纵轴（Auth）承担；把横轴当安全边界的任何论述（例如“Agent 没看到密钥所以安全”）在模型上不成立。	17
7.1	ACB/Fiber 状态迁移图。左：12 状态 ACB（红框为终态）；右：4 状态 Fiber（绿色）。红色虚线为单向耦合 φ ：内核事件可以禁用 Fiber 迁移、强制销毁，但永不启用未被调度器准入的激活；没有 Fiber 事件改变 ACB 状态（命题 7.2）。	19
8.1	工作流 DAG $\rightarrow$ 校验 $\rightarrow$ 运行时管线。上排（蓝）是纯数据变换；下排从触发到运行进入真实世界，但唯一的权威点是每个系统调用处的 Auth。虚线标注被排除的推理路径：定义层命题不能蕴含任何授权结果。	21
9.1	仓库事件 $\rightarrow$ 队列 $\rightarrow$ 调度器 $\rightarrow$ 运行 $\rightarrow$ 证据的端到端管线（本章的排队对象即此管线的 Q_t 与 Run 段）。	23

9.2 排队模型的 理论 曲线 (theoretical queueing illustration, <i>not</i> a measured ConsistenCy benchmark): 接近容量时延迟爆炸, 且被服务时间变异性乘性放大 (Kingman)。LLM 服务时间的实证形状 (令牌量化、共批耦合、历史相关) 见 §9.3。	25
11.1 证据溯源图。快照 (蓝) 由对象库钉定; 分析器带版本读入; 证据身份含 (<i>sha, analyzer, version</i>); 发现经确定性接地分类器 ground 引用证据 (红)。虚线标注被 ground 排除的路径。	30
12.1 证据新鲜度时间线 (理论 示意图)。变更 θ 经检测 (轮询栅格 + 扫描)、防抖 (驻留时间)、运行后产生新鲜证据; $D(t)$ 在此期间阶梯上升、终态后降回。所有量为理论参数, 非实测。	34
13.1 统一形式模型 (PROPOSED_CONSISTENCY_FORMAL_MODEL)。红色为任何策略都不得违背的硬约束; 蓝色目标仅是研究草图——实现的 π 不优化它。	38
17.1 理论支持强度矩阵 (图例: 蓝 =DIRECT, 绿 =STRONG, 琥珀 =USEFUL, 玫瑰 =PROPOSED/N/A, 灰 =WEAK)。逐项论证与文献见第 四-十二章及附录 B。	46

表格

3.1	组件级工程不变式（源码核验摘要）。	7
9.1	排队基线的假设状态表。整体标签： 基线近似 ——诚实的记号是“带批 化到达的 G/G/c 样系统” [24]；本节任何数值都不构成架构不变式。 . .	26
12.1	AoI/新鲜度理论向 ConsistenCy 的迁移账目（怀疑式审计结论）。	36
14.1	形式化性质总表。S = Safety, T = Truthfulness, E = Evidence, R = Reproducibility, L = Liveness。	40
15.1	架构决策 → 理论解释映射表。	42
16.1	与相关系统/论文的比较。LLM Agent 框架一行显示的“无权威中介”正 是 ConsistenCy 的存在理由；与 OTP 的失败哲学分歧（诚实失败 vs 重 启掩盖）见第 五 章。	44
B.1	完整文献映射矩阵。强度列的分级记号（DIRECT、STRONG、USEFUL、 PROPOSED、WEAK、N/A）与正文六类分级一致。	57
C.1	符号表。重载符号（ R_k vs R_t 、数据谓词 φ vs 耦合 φ ）已由上下文与下 标消歧；本表为准。	59

第一章 ConsistenCy v3 是什么

1.1 一句话定位

ConsistenCy v3 是一个仓库原生 (Repository-Native) 的 Agent Harness 操作系统：它以内核化的方式为“多 Agent 持续修改同一仓库”这一场景提供权限、上下文、调度、证据与运行生命周期的统一治理，而公共 PR 审查 (Public PR Review) 只是其上运行的第一个工作负载。系统的物理核心可以写成一个极简公式：

$$\text{ConsistenCy v3} = \text{Kernel} + \text{Cordis Harness} + \text{Evidence Engine}, \quad (1.1)$$

其中 Kernel 负责能力授权 (Capability Broker)、系统调用中介 (Syscall Gateway)、调度 (Kernel Scheduler)、上下文虚拟机 (Context VM) 与审计日志 (Audit Journal)；Cordis Harness 提供响应式的 Fiber 生命周期与共效 (Coeffect) 依赖注入；Evidence Engine 以确定性分析器与内容寻址指纹提供可复现的证据底座。

本报告的研究对象是当前工作树中的真实实现 (分支 v3, 2026-08-28, 含未提交的 CKPT5 变更)，而非任何历史设计稿。所有架构事实均以只读源码调查为准，并给出文件与行号定位；文档与代码不一致之处 (例如能力授权检查“8 步”注释与实际 9 步实现的差异) 一律以代码为准并在正文中如实记录。

1.2 产品要回答的五个问题

ConsistenCy v3 的产品核心价值可以概括为五个问题：

1. 谁被允许做什么 (Who may do what?) ——Kernel Capability 授权；
2. 哪些组件当前应当存活 (Which components should be alive?) ——Cordis Fiber 与共效生命周期；
3. 谁现在运行 (Who runs now?) ——Kernel Scheduler 的准入、并发与预算；
4. Agent 看见什么 (What does the agent see?) ——Context VM 的 ContextImage 与工作集；

5. 什么事实证明结果 (What facts prove the result?) ——Evidence Engine、RepositorySnapshot 与溯源。

这五个问题恰好构成本报告五个理论主题（能力安全、生命周期与授权分离、可见性与权限分离、调度与资源治理、证据与可复现性）的工程原型。换言之，本报告的任务不是给项目“贴数学”，而是把工程上已经成立的边界条件翻译成可以被检查、讨论与继续研究的数学对象。

1.3 本报告的证据纪律

本报告执行严格的分类纪律，任何“理论映射”都必须归入下列六类之一，并在正文与附录 B 的映射矩阵中显式标注：

- DIRECT（直接理论支撑）：该机制是已有理论的标准实例化；
- STRONG（强类比）：理论结构与机制同构，但场景假设不完全重合；
- USEFUL（有用的形式化）：理论提供描述工具而非结论；
- PROPOSED（提议的形式化/扩展）：本报告首次给出的模型，非已确立理论；
- WEAK（弱或投机）：仅作背景，不承担论证责任；
- N/A（不适用）：经研究判定不应映射，作为负结果记录。

所有数学结论同样分类为：THEOREMFROMKNOWNMODEL（已知模型的定理实例）、PROPOSITION（ConsistenCy 模型级命题）、ENGINEERINGINVARIANT（工程不变式，源码可证）、CONJECTURE（猜想）与 PROPOSEDMETRIC（提议度量）。除非明确声明，任何命题证明的都是**架构模型** $\mathcal{M}_C$ 的性质，而不是运行中实现的性质；源码可证的部分会单独标注。报告还区分三类贡献：**既有工程贡献**（项目已实现）、**形式化解释**（本报告首次给出）与**新研究假设**（未来可验证）。

第二章 为什么 Agent Harness 需要理论基础

2.1 问题背景：LLM Agent 的结构性风险

大模型 Agent 系统在 2023–2026 年间快速发展，但主流框架（如以 ReAct [1] 为代表的推理-行动循环、以 AutoGen [2] 为代表的多 Agent 会话编程、以 MetaGPT [3] 为代表的 SOP 编排工作流）普遍将权限问题留给宿主应用的约定，而非系统的构造性属性。经验研究也印证了这一风险的现实性：对大规模真实代码审查评论的实证测量表明，LLM 生成的审查评论中被开发者实际处置的比例呈现强烈的类型依赖性，原始模型输出本身并不是可靠的审查工件 [4]。攻击面研究进一步表明，提示注入（prompt injection）本质上是混淆代理人（confused deputy）问题在 LLM 时代的再现 [5, 6]：拥有权限的代理人被不可信内容诱导，以其环境固有权限（ambient authority）执行违背委托人意图的操作。

ConsistenCy 的核心论点是：**当 Agent 成为仓库的一等修改者时，Agent Harness 必须像一个操作系统内核一样回答权限问题**，而这类问题恰恰是系统安全领域六十余年积累的成熟理论对象——能力系统（capability-based security）[7–10]、引用监视器（reference monitor）[11]、撤销理论 [12]。

2.2 工程直觉为何不够

ConsistenCy v3 的七条架构公理（ $\text{Coeffect} \neq \text{Authorization}$; $\text{Effect} \neq \text{Rollback}$; $\text{Ring} \neq \text{Execution Domain}$; $\text{Cordis Context Isolation} \neq \text{Process Security Isolation}$; $\text{Context VM} \neq \text{Permission System}$; Scheduler 属于 Kernel; Evidence 是 correctness 而非日志）目前以工程断言的形式存在。它们在实践中有效，但存在三个未被回答的问题：

1. **可检查性**：公理之间是否一致？“撤销在下一个系统调用处同步生效”与“Fiber 生命周期异步传播”并存的正确性条件是什么？

2. **可比较性**：缺少形式化描述时，无法将 ConsistenCy 与 seL4 [13]、Capsicum [14]、in-toto [15] 等系统在同一维度上比较，只能停留在术语层面的类比。
3. **可研究性**：调度、新鲜度、证据接地等机制的未来改进（例如预算感知准入、基于版本滞后的触发策略）需要数学模型才能提出并证伪。

2.3 本报告的立场

本报告的立场可以概括为一句话：

找到 *ConsistenCy* 的真实理论基础，并把项目已有的工程直觉转化为可以被检查、讨论和继续研究的数学对象；同时诚实标记所有“看似相关实则不然”的理论，把它们作为负结果记录。

因此，报告包含大量**非主张**（non-claims）：当前调度器不是约束马尔可夫决策过程（CMDP）优化器；信息年龄（Age of Information）对仓库审查新鲜度不是正确的原始度量；当前架构没有经过形式化验证（seL4 意义上）；证据接地在遗留评审 schema 层允许无证据引用的 finding 存在。这些非主张与正面结果同等重要——它们划定了“已确立理论”与“提议模型”的边界，防止理论包装（theory washing）。

第三章 当前系统架构与形式化对象

本章把当前工作树中的 ConsistenCy v3 架构压缩为后续形式化所需的组件清单。每个组件的“工程不变式”都经过只读源码核验（定位到文件与行号），第 四 章起再逐一给出数学对象。

3.1 三层物理结构与形式化对象总览

图 3.1 给出“架构组件 → 理论域”的总映射。每个组件旁标注了本报告采用的主要理论域与支持强度（六类分级见第 一 章）；详细的逐组件论证分布于第 四–十二 章，完整的文献映射矩阵见附录 B。

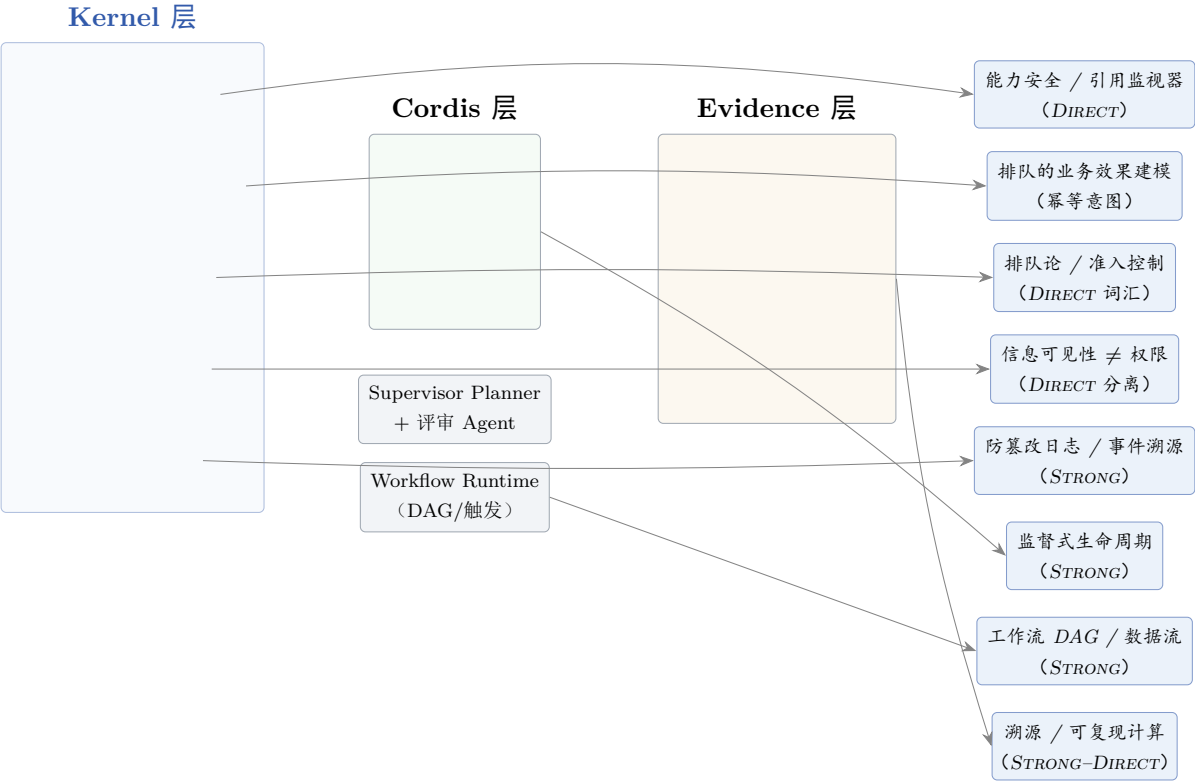


图 3.1: 架构 → 理论域总映射 (ConsistenCy Architecture–Theory Domain Map)。支持强度为六类分级的简写；每条映射的完整论证与关键文献见对应章节及附录 B。

3.2 组件清单与工程不变式

下表汇总源码核验后的 14 个组件及其当前不变式（详细档案见研究笔记；行号针对 2026-08-28 工作树）：

组件	源码核验的不变式（摘要）	主要定位
CapabilityBroker	授权 = 9 个谓词的有序首败拒绝链，每次受保护系统调用都重新求值；撤销/过期单调；允许与拒绝都写审计	capability/broker. ts:245-328
SyscallGateway	intent 类动作（repo.write、github.publish）永不内联分发，必须经 CommitCoordinator； llm.invoke 为 commit/direct，用量由可信 Ring-1 处理器提交	syscall/authorize. ts:46-102
CommitCoordinator	幂等键去重的意图接受器； payloadHash = SHA256(canonical(payload))；内存态，持久性委托给注入的 Sink	commit/coordinator. ts:39-183
KernelScheduler	3 路准入（截止期取消 → 容量 → 优先级-先来先服务，严格平局决断 = 确定性）；终态吸收	scheduler/ scheduler.ts: 225-271
AgentControlBlock	12 状态 LTS；能力引用只存 12 位十六进制指纹（描述符，非凭证）	agent/state.ts: 27-50
Cordis Fiber	PENDING / LOADING / ACTIVE / UNLOADING（Cordis 4.0.0-rc.8 库语义）；共效缺失 ⇒ PENDING；撤销经微任务异步传播	harness-core/ runtime/adapter.ts: 46-130
Context VM	页面不可变、SHA-256 内容寻址；驻留级在覆盖层；pinned→evicted 抛错（失败关闭）；页面持有不授予任何权限	context/manager.ts: 16-19,102-174
AuditJournal	仅 record+entries 接口（无可变性 API）；5 类事件；只出现指纹与哈希，永不出现原始句柄	audit/types.ts: 21-100

组件	源码核验的不变式（摘要）	主要定位
Evidence Engine	指纹 = SHA-256（11 元组规范 JSON）； <code>provenance.sha</code> 参与哈希； <code>store</code> 自行计算指纹、不信任分析器	<code>evidence/</code> <code>fingerprint.ts</code> ： 35-103
RepositorySnapshot	以 <code>rev-parse --verify</code> 失败关闭地钉定 <code>headSha</code> ；读取只经对象库；后续工作树变更不影响既有快照	<code>snapshot/snapshot.ts</code> ：1-120
Workflow Runtime	校验（Kahn）→ 编译（注册表解析，不发放持久授权）→ 运行（逐系统调用授权）；触发是“纯观测溯源，绝非授权”	<code>workflow-runtime/</code> <code>validate, compile, host.ts</code>
触发计划	持久去重键 SHA256[domain, repo, def, event]；围栏式单飞执行；单次尝试，无重试循环	<code>workflow-runtime/</code> <code>triggers.ts, store.ts</code>
仓库监督	轮询 + 观测摘要去重 + 防抖；每个（repo, head, 配置摘要）至多一个事件；只观测记录，不自行入队	<code>heartbeat/</code> <code>repositorySupervisor.ts</code>
LLM 网关	Ring-3 到提供商的唯一路径是逐调用授权的 <code>llm.invoke</code> ；密钥仅服务端；预算是唯一在代码内的反压	<code>workload-review/</code> <code>facades/llm-facade.ts</code> ：73-140

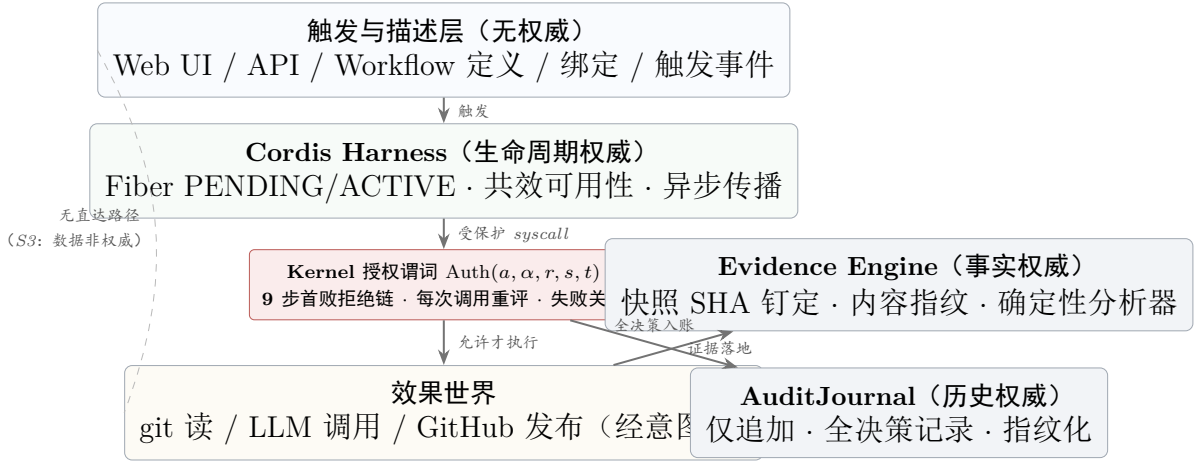
表 3.1：组件级工程不变式（源码核验摘要）。

3.3 权威结构图

图 3.2 展示系统的权威（authority）流向：所有外部效果必须穿过 Kernel 的授权谓词；Cordis 与工作流层只能“触发与描述”；证据层只提供可复现事实。该图是后续所有安全命题（S1-S3）的几何底图。

3.4 形式化对象预览

基于上述事实，本报告在后续章节构造以下数学对象（完整定义见第 四-十三 章，符号表见附录 C）：



- 全局状态 $X_t = (R_t, S_t, Q_t, \mathfrak{A}_t, C_t, V_t, K_t, E_t, P_t, B_t)$ 与演化 $X_{t+1} = \Phi(X_t, u_t, \zeta_t)$ (PROPOSED 统一模型)；
- 授权谓词 Auth 与安全不变式 S1-S3（时序逻辑表述）；
- 可用性 Available、可见性 Visible 与 Auth 的正交性命题；
- 12 状态 ACB LTS 与 4 状态 Fiber LTS 的单向耦合关系；
- 工作流 DAG $G = (V, E)$ 、拓扑序 σ 与“校验 $\neq$ 授权”的裁决面独立性；
- $M/M/c$ 基线排队模型（显式声明的近似）与 CMDP 研究模型（PROPOSED）；
- 证据指纹函数 fp、接地分类器 ground 与确定性/碰撞界命题；
- 版本滞后（Version Staleness）度量 $D(t)$ 与检测时延界（PROPOSED 度量 + STRONG 结构类比）。

第四章 Capability Security 与 Kernel Authority

4.1 理论谱系

ConsistenCy 的能力内核站在一条清晰的理论谱系上。Dennis 与 Van Horn 在 1966 年定义了**能力表 (capability list)**：每个保护域持有一张不可伪造的引用表，每次访问都经内核检查 [7]——这正是 CapabilityBroker 中“能力记录表 + 每次系统调用求值”的直接祖先。Anderson 报告 (1972) 提出**引用监视器 (reference monitor)** 的三条性质：总是被调用 (完全中介)、防篡改、小到可以分析与测试 [11]；Saltzer 与 Schroeder (1975) 随后系统化了最小特权、完全中介、失败默认与机制经济性等设计原则 [8]。Hardy 的**混淆代理人** (1988) 刻画了环境固有权限的危险：被诱导的特权代理人以委托人权限行事 [5]——LLM Agent 的提示注入威胁正是该问题的现代实例 [6]。Miller 等人拆除了能力系统的三个经典迷思 (能力等价于 ACL、不可约束、不可撤销) [9]，其博士论文进一步把对象能力模型与并发控制统一为“健壮组合”理论，其中 vat (事件循环受限的消息传递并发) 模型与 ConsistenCy 的“事件循环串行化内核 + 响应式 Fiber”结构高度同构 [10]。撤销理论方面，Redell 与 Fabry 证明了**经中介间接引用**可实现选择性撤销 [12]；ConsistenCy 的内核记录表正是这个“中介点”，只是它选择了更强的语义——永久单调撤销。在现代系统一侧，seL4 给出了带完整函数正确性证明的能力内核 [13]，Capsicum 展示了能力模式可以在既有通用环境中落地 [14]；二者分别是 ConsistenCy “形似而未证”的金标准与实践先例。

映射强度判定。

- “每次受保护调用重新授权 + 默认拒绝 + 单一权威点” → 引用监视器/完全中介：DIRECT (机制类的标准实例化)；
- “不可猜测的 256 位句柄 + 指纹化审计” → 对象能力的不可伪造引用：DIRECT (以熵而非硬件标签实现不可伪造性，弱于 CHERI 式架构保证，此处如实降格)；
- “永久单调撤销 + 下一调用同步拒绝” → 撤销经中介：DIRECT (Redell-Fabry

问题的退化情形——不需要选择性，因此更易）；

- “LLM 作为不可信提议者” → 混淆代理人/POLA: DIRECT (动机层)；CaMeL 式“能力即数据流控制”为平行现代工作 (STRONG, 未经评审) [6]；
- “两阶段预算预留 (reserve/commit)” → 无已确立理论 (诚实的理论空白；最接近的已发表平行工作是运行时特权策略层)。

4.2 授权谓词

定义 4.1 (授权谓词). 固定主语 a (Agent 身份 $(a.id, a.runId)$)、动作 α 、资源 r (带类别 $\kappa(r)$)、请求作用域 $s = (sha?, paths?)$ 与能力句柄 h 。系统调用请求为四元组 $q = (h, a, \alpha, r, s)$ 。定义九个检查：

$\chi_1 : h \in \text{dom}(C_t)$	(句柄存在)
$\chi_2 : \neg C_t(h).revoked$	(未撤销)
$\chi_3 : C_t(h).expiresAt > t$	(未过期)
$\chi_4 : C_t(h).subject = a.id$	(主语匹配)
$\chi_5 : \kappa(r) \in \{\text{evidence}, \text{workspace}\} \Rightarrow C_t(h).runId = a.runId$	(运行匹配)
$\chi_6 : C_t(h).action = \alpha$	(动作匹配)
$\chi_7 : \text{matchResource}(C_t(h).resource, r)$	(按类别资源匹配)
$\chi_8 : \text{checkScope}(C_t(h).scope, s)$	(作用域匹配)
$\chi_9 : \text{Reserve}_{B_t}(C_t(h), \Delta(\alpha)) = \text{ok}$	(预算预留)

其中 χ_7 按类别定义 (repository 按 id; snapshot 按 repositoryId+sha; evidence/workspace 按 runId; github.publish 可选钉定 pullNumber; llm 按 provider; ast 按 snapshotId); χ_8 中“作用域 sha 已设而请求未带同等 sha”是违规而非豁免，路径经 glob 匹配。授权谓词为有序首败拒绝链

$$\text{Auth}(a, \alpha, r, s, t) \equiv \bigwedge_{i=1}^9 \chi_i, \quad (4.1)$$

在首个失败处返回 $(0, reason_i)$ ；允许与拒绝都写入审计日志。

受保护动作集合 Π 共 12 个注册项，按效果类 $\times$ 分发策略划分：pure/direct {ast.query}; read/direct 六项; revertible/direct {workspace.write, evidence.write}; commit/direct {llm.invoke}; commit/intent $\Pi_{\text{intent}} = \{\text{repo.write}, \text{github.publish}\}$ ——后者永不内联分发，只能经 CommitCoordinator 的幂等意图门。

假设 A1 (完全中介 (broker 层)). Π 中每个效果都经 `gateway.invoke` 到达 `broker.authorise`。 **边界条件**: 网关对未注册动作是穿透的 (网关层无显式未知系统调用拒绝) ; 中介完备性依赖 broker 的 χ_1 与发放策略拒绝——发放校验 `CAPABILITY_ISSUABLE_RING`, 未注册动作不存在记录, χ_1 或 χ_6 必然失败。复合系统仍为默认拒绝, 但**完全中介者是 broker, 不是网关**。

假设 A2 (同步串行化中介). `Auth` 在系统调用边界内同步求值; 内核中介循环内的调用被宿主事件循环全序化; χ 求值与处理器调用之间不会交错撤销写 (网关检查内部无 `TOCTOU` 窗口)。注意该全序声明只覆盖内核中介循环——执行域中的 `worker` 线程/子进程代码不受其约束。

假设 A3 (无缓存). 九个检查在每次调用上重新求值; 任何决策不跨系统调用缓存。

`TOCTOU` (time-of-check to time-of-use) 正是 A2/A3 所防御的漏洞类: 名字与对象的绑定可能在检查与使用之间改变 [16]。ConsistenCy 的对策是三重的: 逐调用重评 (A3)、检查与执行的中介点串行化 (A2)、以及作用域钉定内容 (sha) 而非可变名字。

4.3 安全不变式 S1

记 $\text{Exec}(\alpha, r, t)$ 为“请求 (α, r) 的处理器在步 t 产生效果”; 每次执行事件携带其中介点 $m(q)$ ——同一 `gateway.invoke` 内 `Auth` 被求值的串行化步, $m(q) \prec_{\text{ser}} t$ 。

命题 4.2 (安全不变式 S1: 边界授权). 在假设 A1–A3 下,

$$\mathbf{S1} : \quad \square(\text{Exec}(\alpha, r, t) \Rightarrow \text{Auth}(q, m(q)) = 1). \quad (4.2)$$

证明梗概 (构造性). 在 $\mathcal{M}_C$ 中, 产生 `Exec` 事件的唯一迁移规则是“`invoke` 求值 `Auth`; 为 0 则抛出 `CapabilityError` (处理器不被调用); 为 1 才进入处理器” (`syscall/authorize.ts:46--102`; `intent` 类在授权之前即抛出, 内联处理器调用数为零)。若 A1–A3 任一失守, 该规则就不再是执行事件的唯一生产者, S1 变为空洞。 $\square$

[ENGINEERING_INVARIANT (代码可证调用路径; “不存在旁路”是对全部未来代码的全称否定, 属假设)]

失败条件. (i) 绕过 `invoke` 的效果路径 (A1 失守; 注意 OS 级文件系统/网络围堵在当前实现中除子进程隔离外未强制, S1 对模型外效果不做声明); (ii) 跨边界缓存 (A3); (iii) 非串行化运行时中的检查–使用间隙 (A2 失守时 S1 在中介点语义下仍成立, 但更强的“ t_r 后无执行” (S2) 需要 A2); (iv) 网关穿透本身不是 S1 的违反 (broker 拒绝), 但把完备性负担集中在单一组件上。

4.4 撤销支配 (S2)

假设 **A4** (标志单调). C_t 的迁移规则永不复位 *revoked* (撤销永久——代码可证, `broker.ts:195--218`)。

假设 **A5** (原子撤销写). 对 $C_t(h^*).revoked$ 的写相对于中介求值是原子的 (事件循环串行化, 与 A2 同基础设施)。

命题 **4.3** (撤销支配). 设 $revoke(h^*, t_r)$ 为 $revoked : \perp \rightarrow \top$ 的迁移 (日志 + 事件总线; 无级联; 向 *Cordis* 的传播延迟到微任务队列)。则对每个引用 h^* 且 $m(q) \succeq_{\text{ser}} t_r$ 的请求 q :

$$\text{Auth}(q, m(q)) = 0, \quad \text{与 } K_{m(q)}, V_{m(q)}, P_{m(q)}, \mathfrak{A}_{m(q)}, Q_{m(q)} \text{ 无关.} \quad (4.3)$$

证明. $\text{Auth} = \bigwedge_i \chi_i \leq \chi_2$ 。由 A4, 对一切 $t \succeq t_r$ 有 $revoked = \top$, 故 $\chi_2 = 0$, $\text{Auth} = 0$ (拒绝原因为 *revoked*, 链上第 2 位)。无关性是语法性的: Auth 的自由变元含于 $\{C_t, B_t, h, a, \alpha, r, s, t\}$; $K, V, P, \mathfrak{A}, Q$ 不出现在公式中, 而 Auth 是其声明输入的全函数, 故排除分量的任何变动不改变裁决。结合 S1: 不存在中介点 $\succeq t_r$ 的执行。□

[PROPOSITION (模型级; A_4/A_5 代码可证)]

意图门 carve-out (如实声明)。对 $\alpha \in \Pi_{\text{intent}}$, 授权发生在**意图接受时**: 接受之后的撤销不抹除已接受意图; 不可逆外部效果在幂等键去重与下游租约/退避/重试 (租约 30s、超时 15s、最多 3 次尝试) 之下继续执行。形式上 $\text{Accept}(key) \Rightarrow \text{Effect}$ 由接受器而非效果时刻的 Auth 支配。对称地, 接受时刻同样“冻结”了 χ_3 (过期) 与预算预留的判定。因此 S2 的正确读法是**作用于中介点**; 若读作“撤销后不再发生任何来自撤销前已接受意图的不可逆效果”, 则该命题对 *intent* 类为假。

在飞操作。中介点先于 t_r 的操作会完成——撤销不是对在飞副作用的取消, 只是对后续授权的取消。这是“ $\text{Effect} \neq \text{Rollback}$ ”公理 (见第 五 章) 在时序上的体现。

4.5 撤销时间线

图 4.1 展示撤销支配与异步传播的并存: 内核裁决 (黑) 在 t_r 后立即转为拒绝; Fiber 生命周期 (灰) 要等到 t_{unload} 才完成卸载; 两者之间的陈旧门面窗口 W_{stale} 内, 任何经陈旧门面发起的系统调用都被拒绝 (命题 4.3), 窗口长度 $\delta_{\text{prop}} \geq 0$ 甚至可以是 ∞ 而不影响安全性。

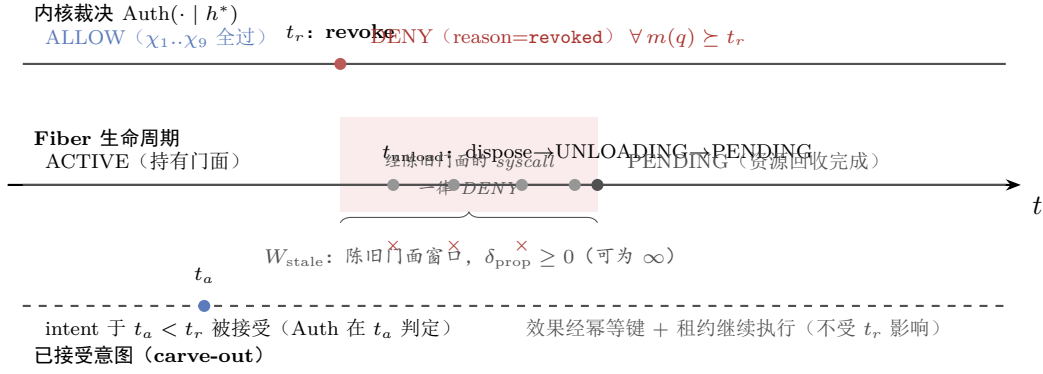


图 4.1: 能力授权时间线与撤销。上: 内核裁决在 t_r 后同步转为拒绝 (命题 4.3); 中: Fiber 卸载异步完成, 窗口 W_{stale} 内的调用全部被拒; 下: 撤销前已接受的意图不受影响 (carve-out)。

4.6 与经典理论的差异点

与 Redell-Fabry 的选择性撤销相比, ConsistenCy 选择了更简单的语义: 撤销是全局永久的, 不需要“看守人 (caretaker)”间接层来隔离单个持有者; 这换来的是命题 4.3 的单调性证明极其简短。与 seL4 相比, ConsistenCy 没有任何机器检查的证明——S1/S2 是“模型级命题 + 源码核验的构造性论据”, 其“不存在旁路”部分 (A1 的全称否定) 恰恰是形式化验证 (如 Isabelle/HOL 之于 seL4 [13]) 才能消除的假设负担; 这构成本报告在第 十九 章中声明的核心局限之一, 也是后续模型检验扩展 (§7.3) 的动机。

第五章 Cordis Lifecycle 与 Authorization 的分离

5.1 工程事实

Cordis 层 (Cordis 4.0.0-rc.8 库语义之上) 管理响应式组件的生命周期: Fiber 状态为 PENDING / LOADING / ACTIVE / UNLOADING; 共效 (Coeffect) 以声明式依赖 (如 `Plugin.Function.inject=["ast"]`) 表达 “进入 ACTIVE 所需的服务”; 每个 Agent 经 `root.isolate` 获得隔离的依赖上下文; 内核能力经 `CapabilityLifecycleAdapter` 桥接为服务门面 (facade)。关键事实是两个时间尺度:

- 能力撤销后, `capability.revoked` 事件在微任务队列上调度销毁器; 在传播完成前, Fiber 可以保持 ACTIVE 并持有陈旧门面;
- 但该陈旧门面在下一次系统调用处即被内核拒绝 (命题 4.3)。

即: 撤销在生命周期层是最终一致的, 在授权层是同步的。组件公理 “安全性不依赖传播时序” 由此获得精确含义。

5.2 理论映射

Cordis 层在 Harel-Pnueli 的分类下是一个响应式系统 (reactive system) ——持久运行、事件驱动、与环境持续交互而非计算到终态 [17]; 这一定位把 “Fiber 生命周期为何需要独立于任何单次运行来推理” 变得自然。Fiber 的监督式生命周期 (依赖缺失即不激活、失败即清理、可被监督者处置) 在理论上最接近两条线: 其一是 Actor 模型的严格语义 [18] 与 Erlang/OTP 的监督树 (supervision tree) ——Armstrong 的论文把 “错误内核 vs 错误携带” 与分层重启发展为可靠分布式系统的构造原理 [19]; 其二是 Miller 的 vat 模型: 事件循环受限的响应式并发与访问控制的统一 [10] (vat 的技术核心之一是 vat 间消息传递的全序保证; ConsistentCy 的对应物是内核中介循环的串行化——假设 A2——但 Cordis 层并未声明跨上下文的端到端消息序, 故本类比限定

于“事件循环受限的并发单元 + 无环境固有权限”这一结构层)。**映射强度: Strong**。必须指出的分歧点: OTP 的监督哲学是“让它崩溃、以重启掩盖故障”; ConsistenCy 的触发计划执行是**单次尝试 + 诚实的重启失败记录**(执行中断在启动恢复时被标记为 `failed` 而非静默重试)——这是同一监督传统内的另一个设计点, 选择“失败关闭的诚实”而非“尽力掩盖的可用性”。该分歧不能被表述为“ConsistenCy 实现了 OTP 监督语义”, 只能表述为“处于同一监督谱系”。

5.3 可用性与授权的正交性

定义 5.1 (可用性). $\text{Available}(a, s, t) \equiv [\text{共效服务 } s \text{ 在 Agent } a \text{ 的隔离上下文中已提供}] \wedge [a \text{ 的 Fiber 处于 ACTIVE}]$ 。

命题 5.2 (可用性 $\not\vdash$ 授权, 反之亦然). $\text{Available}(a, s, t) \not\vdash \text{Auth}(a, \alpha, r, s, t)$ 且 $\text{Auth} \not\vdash \text{Available}$ 。且安全性只依赖 Auth : 若 $\text{Available} \equiv 0$, 则无任何执行发生, $S1$ 空洞成立; 可用性失败只降低活性(效果无法完成、Fiber 停留 `PENDING`、Agent 甚至无法发起调用), 永不授予权威。

见证结构. $\mathcal{W}_1$ (授权 $\wedge$ 不可用): C_t 有有效未撤销的 (`ast.query`, `snapshot`) 记录, 但 `ast` 服务从未提供——Fiber 停留 `PENDING`, Agent 永远无法调用; 假想请求的 $\text{Auth} = 1$, 而 $\text{Available} = 0$ 。 $\mathcal{W}_2$ (未授权 $\wedge$ 可用): Fiber `ACTIVE` 持有门面, $t_r < t$ 时底层能力被撤销而传播未运行; $\text{Available} = 1$, $\text{Auth} = 0$ (命题 4.3)。两个见证各自满足一个合取而证伪另一个, 故两个蕴含都不有效。 $\square$

[PROPOSITION]

命题 5.3 (传播时序无关性). 对一切 $\delta_{\text{prop}} = t_{\text{unload}} - t_r \in [0, \infty]$ (含 ∞ , 即销毁器永不运行): 窗口 W_{stale} 内经陈旧门面发起的全部系统调用都被拒绝。 δ_{prop} 的上界(生命周期的有界最终一致性)只对资源回收——一个活性/整洁性质——是必需的, 对安全健全性永不需; 传播饥饿(微任务队列拥塞)打不开任何权威窗口。

证明. 窗口内每个系统调用的中介点 $\succeq t_r$; 应用命题 4.3。 $\square$

[PROPOSITION]

第六章 Context Visibility 与 Authority 的分离

6.1 工程事实

Context VM 是“虚拟上下文内存”：不可变的 `ContextPage` 以 SHA-256 内容哈希寻址；驻留级（`pinned/hot/cold/evicted`）位于镜像覆盖层而非页面；工作集 = `pinned+hot`；除 `pinned→evicted` 抛错（失败关闭）外驻留级可自由迁移；COW fork 只复制页面表、共享页对象；渲染顺序由类别先序与页 id 决定（绝不依赖哈希表插入序）。源码中明文写明：“Context VM 是状态管理，不是授权；页面成员资格不授予任何能力”（`context/manager.ts:16--19`）。当前实现没有自动逐出、摘要或检索策略——任何暗示存在此类策略的描述都是过度陈述。

6.2 可见性与权限的正交性

定义 6.1 (可见性). $\text{Visible}(a, x, t) \equiv \text{PT}_a(x) \in \{\text{pinned}, \text{hot}\}$ （页 x 在 a 的工作集中）。

命题 6.2 (可见性 $\not\vdash$ 授权，反之亦然). V_t 不是 `Auth` 的输入（定义 4.1 的自由变元不含 V_t ）。见证：页在而无能力（“页面成员资格不授予能力”）给出 $\text{Visible} \wedge \neg \text{Auth}$ ；有 `repo.read` 能力而相关页全部被逐出到 `cold` 给出 $\text{Auth-可能} \wedge \neg \text{Visible}$ 。

[PROPOSITION]

6.3 为什么这不是语义游戏

图 6.1 的正交结构排除了两类常见的错误论证：

1. “看不到即安全”：即使某秘密页从未进入 Agent 工作集，也不能据此声称机密性——那需要信息流不干涉类的性质，而本系统并未声明（密钥的机密性由服务端持有与脱敏保障，是另一条防线，不属于 `Visible / Auth` 轴）；

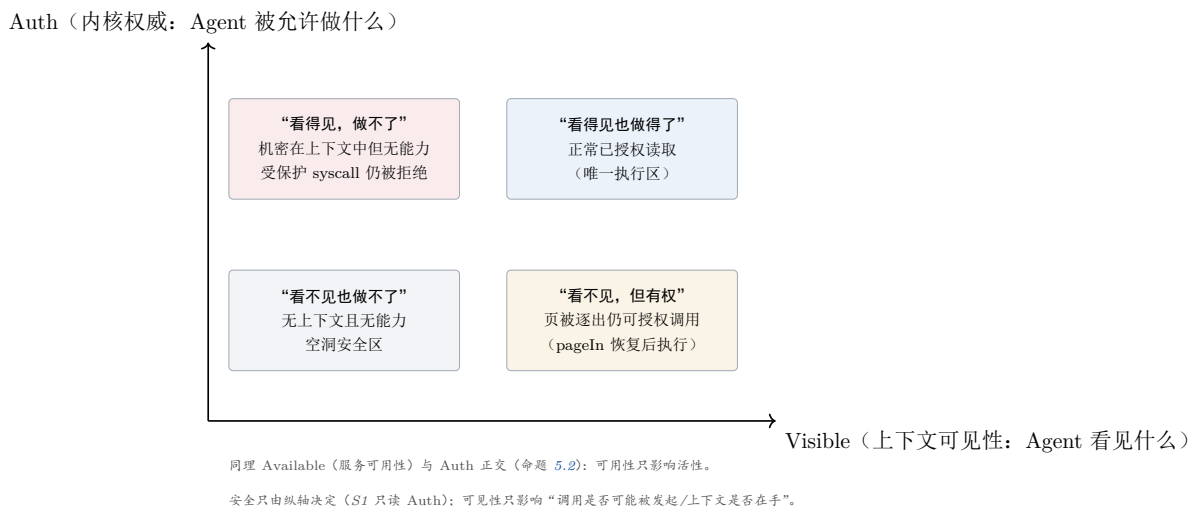


图 6.1: 上下文可见性与权威的正交轴。Context VM $\neq$ 权限系统: 两轴独立变化, 安全性只由纵轴 (Auth) 承担; 把横轴当安全边界的任何论述 (例如 “Agent 没看到密钥所以安全”) 在模型上不成立。

2. **“看到了就危险”:** 页在上下文中不提升任何权威——受保护系统调用照样要过 9 步链。反过来, Visible 的驱逐/钉定只影响性能与正确性 (如 `pinned`→`evicted` 失败关闭保护关键上下文不丢失), 不影响权限。

这与 Saltzer-Schroeder 的“关注点分离/机制经济性”一脉相承 [8]: 可见性 (内存语义) 与权威 (保护语义) 是两个正交的轴, 各自的最优设计互不干扰。值得注意的是, COW fork 的“父修改不流入既有子镜像”与内容寻址一起, 使 Visible 成为一个纯函数式的投影——渲染是 (页表, 驻留级, 先序) 的确定性函数, 这为第十一章的可复现性论证提供了上下文侧的对应物。

第七章 Agent / Run / Fiber 状态模型

7.1 三个状态机，三种所有者

ConsistenCy 的运行由三个显式状态机组成，分属不同所有者：

1. **ACB (Agent Control Block) LTS**: 12 状态，由 KernelScheduler 独占写入。状态集 S_{ACB} 含 12 个状态：NEW、READY、RUNNING、五个等待态 (WAIT_LLM / WAIT_TOOL / WAIT_IO / WAIT_AGENT / WAIT_HUMAN)、SUSPENDED 与三个终态；终态集 $T_{term} = \{SUCCEEDED, FAILED, CANCELLED\}$ 吸收（迁移表 `agent/state.ts:27--50`）。执行器/Agent 代码只能请求阻塞与唤醒（`RUNNING`→`WAITr` 与 `WAITr`→`READY` 的原因），且必须经调度器原语生效；网关不拥有任何 ACB 迁移（其拒绝结果作为执行器层的失败请求传播）。
2. **Run LTS**: `CREATED`→`ACTIVE`→ 终态；运行取消级联到所有非终态 Agent；已派发的外部工作不被回滚。
3. **Fiber LTS**: 4 状态 `PENDING` / `LOADING` / `ACTIVE` / `UNLOADING`，语义由 Cordis 库拥有（外部公理；升级 Cordis 版本会重新打开本章结论）。

理论上，ACB 的 12 状态迁移表就是 Keller 意义下的带标迁移系统 (labelled transition system, LTS) [20]；对迁移不变式（“没有任何迁移离开 `SUCCEEDED`”、“`WAITr` 最终恢复或失败”）的自然语言是 Pnueli 的时序逻辑 [21]。映射强度：DIRECT（不是类比——它字面上就是有限 LTS）。

7.2 单向耦合积

定义 7.1 (单向耦合 φ)。部分函数 φ 把调度器标签映射为 Fiber 标签： $\varphi(\text{admit}) = \text{activate}$, $\varphi(\text{suspend}) = \text{cleanup}$, $\varphi(\text{cancel}) = \text{dispose}$ ；内核事件 `revoke` 自主触发 `dispose`。方向性：内核事件可以禁用 Fiber 迁移（撤销禁用后续激活并强制销毁），但永不启用任何调度器未准入的激活；没有任何 Fiber 事件改变 ACB 状态。

命题 7.2 (积的良好定义与派生不变式). φ -受限异步积 $\mathcal{T} = \mathcal{T}_{ACB} \parallel_{\varphi} \mathcal{T}_F$ (两迁移关系的交错, *Fiber* 激活/清理标签须与其 φ -原像共现, *dispose* 允许自主发生) 是良定义 *LTS*: φ 是不相交标签集之间的部分函数, 复合迁移关系在每个复合标签处单值。状态空间 $\subseteq S_{ACB} \times S_F$ (48 对)。由桥接映射派生: *Fiber ACTIVE* $\Rightarrow$ *ACB* $\in \{RUNNING\} \cup \{WAIT^{\tau}\}$; 故 (*WAIT_LLM*, *ACTIVE*) 可达 (已证实), 而 (*NEW*, *ACTIVE*) 与 (*SUSPENDED*, *ACTIVE*) 不可达。

证明. φ 的部分性与函数性保证合成约束无歧义; 不可达性由 φ -共现要求直接得出 (*NEW* 与 *SUSPENDED* 不出现在任何 φ 定义域原像的使能条件中)。□

[PROPOSITION (*LTS* 定义为 *ENGINEERING_INVARIANT*; 积性质为模型级)]

(*WAIT_LLM*, *ACTIVE*) 的可达性值得强调: 它说明 *Fiber ACTIVE* $\neq$ 调度器 *RUNNING* 是系统的设计事实而非缺陷——*Fiber* 反映 harness 侧的组件存活, 调度器跟踪协作式等待。这一分离与第 五 章的正交性共同构成“安全看内核、活性看生命周期”的二分。

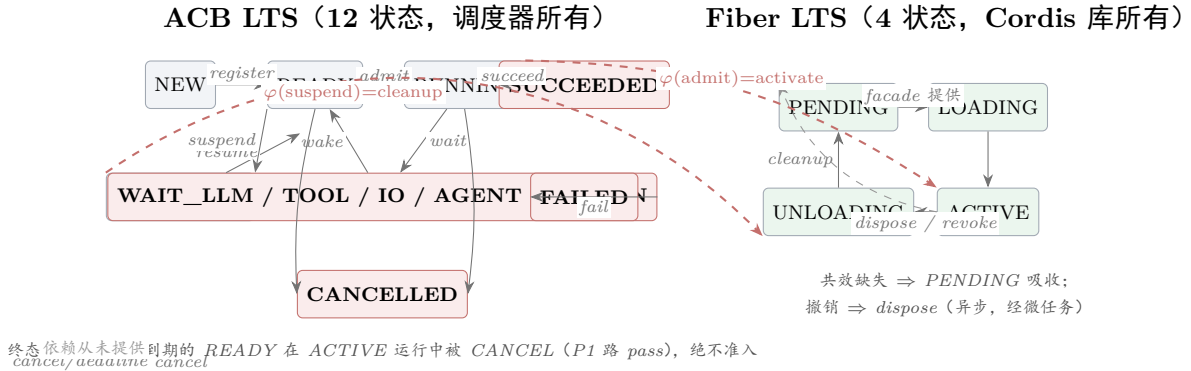


图 7.1: ACB/Fiber 状态迁移图。左: 12 状态 ACB (红框为终态); 右: 4 状态 Fiber (绿色)。红色虚线为单向耦合 φ : 内核事件可以禁用 Fiber 迁移、强制销毁, 但永不启用未被调度器准入的激活; 没有 Fiber 事件改变 ACB 状态 (命题 7.2)。

7.3 模型检验扩展 (Proposed)

48 对的有限积空间提示了一个尚未接线的验证路径: 把 S1-S3 与积 LTS 的不变式编码为 CTL 公式并模型检验——这正是 Clarke-Emerson 分支时序逻辑模型检验的诞生场景 [22]。当前实现以测试钉住若干不变式 (终态吸收、重复入队防御、不可达组合), 但未做穷举检验。该扩展不改变任何结论的效力, 只是把“逐条断言”升级为“穷举排除”; 列入第 二十 章的研究议程。

第八章 Workflow DAG 与执行语义

8.1 四段管线

workflow运行时把一个经验证的定义变成一次受治理的运行，管线为四段（全部源码核验）：

1. `validate : Def → Def ∪ {⊥}`——结构良构性校验，含以 Kahn 入度消解实现的环检测；成功时给出确定性拓扑序 σ （同层平局按节点 id 决断）；
2. `compile : Def ×  $\mathcal{R}$  → Plan ∪ {⊥}`——节点类型对运行时注册表 $\mathcal{R}$ 解析（当前两类：`analyzer.deterministic-evidence` 需 `{repo.read, evidence.write}`；`verifier.persisted-evidence` 需 `{evidence.read}`）；能力需求必须是已注册系统调用动作；编译器不发放任何持久授权——需求是数据；
3. `dryload`——逐节点可行性复查（复用同一编译函数；明示“仅可行性检查”）；
4. `execute`——`host.trigger` 在任何运行记录之前失败关闭（未知仓库 404 / 快照不可得 503 / 定义不可执行 409）；钉定 HEAD 后创建快照；持久化运行后异步执行；节点内逐系统调用授权（第 四 章）。

定义携带字面量 `failurePolicy="fail-closed"`：节点失败产生终态运行状态，而非部分继续。

8.2 拓扑序定理与 WF-net 负结果

定理 8.1 (拓扑序存在性). 无环有向图 $G = (V, E)$, $|V| \geq 1$, 存在拓扑序 $\sigma : V \rightarrow \{1..|V|\}$ 使 $(u, v) \in E \Rightarrow \sigma(u) < \sigma(v)$ ；Kahn 算法以 $O(|V| + |E|)$ 时间计算一个拓扑序（或检出环）。

[*THEOREM_FROM_KNOWN_MODEL*（算法课标准结论；实现即 `validate.ts` 的入度消解）]

Petri 网/WF-net 负结果 (N/A-for-now)。 workflows 理论的标准形式化是 van der Aalst 的工作流网 (WF-net) 及其健全性 (*soundness*): 正常完成可达、无死任务、无遗留托肯 [23]。经评估, 对当前的 ConsistenCy 语义——无环 DAG、确定性平局决断、无延迟选择 (deferred choice) 与里程碑——Kahn 拓扑序已经给出终止性与无死锁的构造性保证, WF-net 的位置不变量与托肯演算 **不增加证明力**。该映射记为负结果: 完整 Petri 机制推迟, 仅当语言将来引入环、迭代或延迟选择时才值得引入。这不是对 WF-net 理论的否定, 而是对其适用边界的诚实划定。

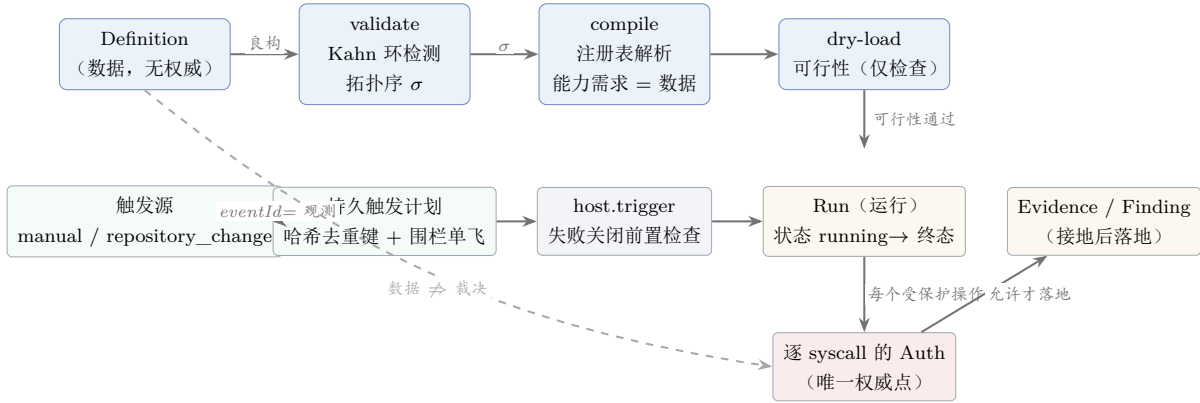


图 8.1: 工作流 DAG → 校验 → 运行时管线。上排 (蓝) 是纯数据变换; 下排从触发到运行进入真实世界, 但唯一的权威点是每个系统调用处的 Auth。虚线标注被排除的推理路径: 定义层命题不能蕴含任何授权结果。

8.3 裁决面独立性 (“校验 ≠ 授权”)

命题 8.2 (裁决面独立性 (单步、固定状态)). 把状态分解为 $X_t = (D_t, Y_t)$, 其中 D_t 是数据面坐标 (定义、编译计划、绑定、触发模式、事件标识), $Y_t \supseteq (C_t, B_t)$ 。裁决泛函 $\mathcal{V}: (q, X) \mapsto \text{Auth}(q)$ 在数据替换下不变: 对一切 D, D' ,

$$\mathcal{V}(q, (D, Y)) = \mathcal{V}(q, (D', Y)). \quad (8.1)$$

因此, 任何局限于同一时刻求值的数据面命题 φ 都不能蕴含任何非平凡的授权结果 (除非 φ 本身不可满足或恒真)。

证明. 由定义 4.1 的语法: Auth 的自由变元不含任何 D -坐标。见证: (i) 合法计划 + 零能力发放 $\Rightarrow$ 运行内所有请求被拒 (χ_1 失败); (ii) 无计划 (手动运行) + 内核发放的能力 $\Rightarrow$ 请求被允许。校验对任何授权结果既不必要也不充分。 $\square$

[PROPOSITION (安全不变式 $S3$ 的形式内容)]

适用范围（对抗误读）。本命题不是轨迹层的不干涉（noninterference）：数据面的授权标志（catalog 投射到 `ReviewWorkload.issueCapabilities`）因果地决定哪些能力被申请发放，从而影响后续裁决的流。声明的、且仅声明的是：发放本身是内核侧经 Ring 策略校验的操作（Ring 0 动作不能在内核外发放），因此“数据 → 权威”的唯一合法通道是内核侧的 Ring 校验发放；对任何给定请求的裁决不随数据面直接变化。触发事件是“纯观测溯源，绝非授权”（`triggers.ts`）。

8.4 执行健全性与活性

命题 8.3 (执行健全性与活性). **健全性**：运行的每个外部效果要么 (a) 是中介点处 $\text{Auth} = 1$ 的中介系统调用 ($S1$ 在工作流层的实例化)，要么 (b) 是经幂等、哈希承诺、持久 Sink 落地的已接受提交意图。**活性**：在 (F1) G 有限无环；(F2) 节点执行有限步内终止（确定性分析器全函数；LLM 路径一次修复尝试后类型化失败）；(F3) *fail-closed* 使节点失败产生终态；(F4) 公平派发（期望性假设：单飞、5s 轮询、批量 5、单次尝试、无重试；高优先级干扰可饿死）；(F5) 无崩溃（启动恢复把 *executing* 标记为 *failed*——诚实失败而非进展）之下，运行按 σ 序在有限步内到达终态。

[健全性 *ENGINEERING_INVARIANT* (逐路径代码可证)；活性 *PROPOSITION* (F4/F5 显式为期望性假设；F2 的提供商挂起已并入 P_i 可用性假设)]

诚实限制（如实编码的实现边界）。

- 触发是“恰好一次发起、至多一次执行”：持久去重键 `SHA256[domain, repo, def, event]` 与围栏式 `UPDATE WHERE status='pending'` 保证每个事件每个定义至多发起一次运行；但计划执行单次尝试、无重试循环——鲁棒性以“状态转换间崩溃”为界，不含运行中挂起。
- **CommitCoordinator** 的去重态在内存：未配置持久 Sink 的重启会丢失去重记忆，重启后重复接受同一幂等键成为可能（“恰好一次发起”以 Sink 持久性为条件）。
- 监督侧事件去重也在内存：事件标识仅经派生键持久。

这三条都不是缺陷声明，而是把“什么在什么故障模型下成立”写清楚的边界条件；对应的研究问题（恰好一次效果发起的可用形式化）见第 二十 章。

第九章 排队、调度与资源治理

9.1 调度器的真实形状

KernelScheduler 的准入是三路串行判定（源码核验，`scheduler.ts:225--271`）：

1. **P1 截止期执法**：ACTIVE 运行中已过期（ $now \geq deadline$ ）的 READY Agent 被取消而非准入；
2. **P2 容量守卫**： $runningCount \geq c$ （ $c = \maxRunningAgents$ ，当前 workflow 执行器接线为 $c = 1$ ）则不准入；
3. **P3 优先级-先来先服务**：最大 $priority$ 者胜出，严格 $>$ 平局决断 = 同优先级内 FIFO——确定性准入。

远程 LLM 推理**不可物理抢占**：调度器的真实模型是“准入控制 + 协作调度 + 预算/并发控制”，而不是抢占式 CPU 调度器。这一形状声明直接决定了本章理论的适用边界。

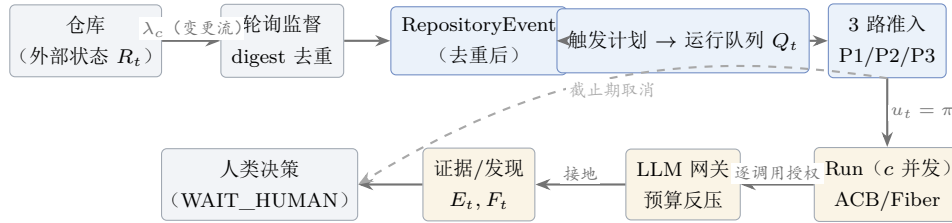


图 9.1: 仓库事件 → 队列 → 调度器 → 运行 → 证据的端到端管线（本章的排队对象即此管线的 Q_t 与 Run 段）。

9.2 排队基线模型（基线近似）

本节全部内容是**显式声明**的简化：以 Kendall 记号 [24] 的词汇做容量级推理。设 λ 为评审任务到达率、 μ 为单服务器完成率、 $\tau = 1/\mu$ 、 $A_e = \lambda/\mu$ 、 $\rho = \lambda/(c\mu)$ 。

定理 9.1（稳定性（ $M/M/c$ 理想化的定理））。 $\lambda < c\mu$ 是 $M/M/c$ 正常返的充要条件 [25]。

定理 9.2 (Erlang-C 等待 ($M/M/c$ 理想化的定理)). 到达需等待的概率与均值等待:

$$P_{\text{wait}} = \frac{\frac{A_e^c}{c!(1-\rho)}}{\sum_{k=0}^{c-1} \frac{A_e^k}{k!} + \frac{A_e^c}{c!(1-\rho)}}, \quad \mathbb{E}[W_q] = \frac{P_{\text{wait}}}{c\mu - \lambda}, \quad \mathbb{E}[W] = \mathbb{E}[W_q] + \tau. \quad (9.1)$$

$c = 1$ 时退化为 $P_{\text{wait}} = \rho$ 、 $\mathbb{E}[W_q] = \rho/(\mu - \lambda)$ [25]。

定理 9.3 (Little 定律). $L = \lambda W$ 与 $L_q = \lambda W_q$: 平均在队数 = 到达率 $\times$ 平均逗留时间, 与排队纪律无关 (优先级下同样成立) [26]。对 *READY* 积压的工程含义: 测得 L 与 λ 即可估计 W , 无需解整个系统。

定理 9.4 (Kingman 重交通近似 ($GI/G/1$ 理想化的定理)). 单服务器情形 ($c = 1$; 非等式, 且不直接适用于 $c > 1$):

$$W_q \approx \frac{\rho}{1-\rho} \cdot \frac{c_a^2 + c_s^2}{2} \cdot \tau, \quad (9.2)$$

c_a^2, c_s^2 为到达间隔与服务时间的平方变异系数; $M/M/1$ ($c_a^2 = c_s^2 = 1$) 时精确 [27]。信息: 接近容量时延迟随变异性成倍膨胀。

定理 9.5 (非抢占 HOL 优先级 ($M/G/1$ 类理想化的定理)). 优先级类 $1 > 2 > \dots$, 类内 *FIFO* (即 $P3$ 的形状)。记 $R_k = \sum_{i \leq k} \rho_i$ 、 $W_0 = \sum_i \lambda_i \mathbb{E}[S_i^2]/2$, 则

$$\mathbb{E}[W_q^{(k)}] = \frac{W_0}{(1 - R_{k-1})(1 - R_k)} \quad (9.3)$$

(*Kleinrock* 非抢占优先级均值等待 [25])。饥饿形状: 当高类饱和 $R_{k-1} \rightarrow 1$ 时 $\mathbb{E}[W_q^{(k)}] \rightarrow \infty$ ——即使总体 $\rho < 1$; 低优先级均值延迟以 $(1 - R_{k-1})^{-1}$ 爆炸。公平份额替代 (逐流隔离、加权份额) 在文献中成熟 [28], 但未实现——*ConsistenCy* 以确定性严格优先级换取简单性, 把低类饥饿留给截止期取消兜底。

图 9.2 的系特异性推论 (使其非装饰性)。 排队曲线本身是教科书结论; 把它接入 *ConsistenCy* 的实际架构可以得到两条可行动的推论。其一, 到达结构偏离泊松是架构决定的, 容量余量必须显式计入 c_a^2 : 监督管线对变更流做了三重变换——轮询栅格量化检测、防抖冲刷 (检测时刻 $+ T_{\text{deb}}$, 离栅格连续变化)、突发合并 (去重把一次观测窗口内的多次变更折为至多一个待执行计划)。前两者在单仓库稀疏情形反而降低到达间隔的变异 (趋于规则化), 第三者则在高峰期同时降低有效到达率与增大到达的间歇性——三个效应方向不一, 任何单向断言 (如 “批化必然使 $c_a^2 > 1$ ”) 都不成立; 正确结论是 c_a^2 是由 $T_{\text{poll}}, T_{\text{deb}}$ 与合并策略共同决定的、可测的量, 容量规划不应假设它为

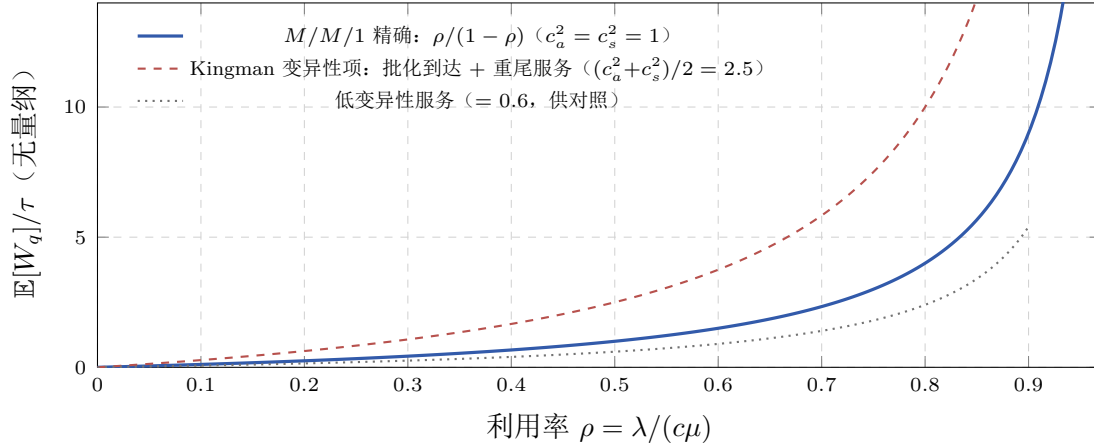


图 9.2: 排队模型的理论曲线 (theoretical queueing illustration, *not* a measured ConsistenCy benchmark): 接近容量时延迟爆炸, 且被服务时间变异性乘法放大 (Kingman)。LLM 服务时间的实证形状 (令牌量化、共批耦合、历史相关) 见 §9.3。

1. 其二, **Kingman 项给出参数化的余量规则**: 若目标是 $\mathbb{E}[W_q] \leq \kappa \cdot \tau$ (平均等待不超过 κ 个服务时间), 由定理 9.4 解 $\frac{\rho}{1-\rho} \cdot \frac{c_a^2 + c_s^2}{2} \leq \kappa$, 记 $S = c_a^2 + c_s^2$, 得

$$\rho \leq \frac{2\kappa}{S + 2\kappa}, \quad (9.4)$$

即容许利用率随 S 递减——这是从该图直接读出的、以 (κ, S) 为参数的容量上界规则: 给定实测的 S , 它能立即回答“ $c = 1$ 的接线最多安全承载多大的 λ/μ ”。

9.2.1 假设表 (含“违背”列)

假设	在 ConsistenCy 中的状态
Poisson 到达	违背 。到达是轮询批化的: 事件在轮询栅格时刻发射 (每个 (repo, head, 配置摘要) 每次观测至多一个), 再经防抖冲刷与去重削减; 多仓库叠加趋向 Poisson, 但少仓库时栅格结构主导。
指数服务	违背 。LLM 服务时间被输出令牌数量化 (块状)、随共批负载耦合 (非独立) [29]、随服务器内部状态漂移 (客户端不可见) [30]。既非独立同分布也非指数。
类内 FCFS	成立 。P3 严格平局 FIFO, 代码可证的确定性。
单服务器 “一次一个在服务”	部分违背 。RUNNING $\rightarrow$ WAIT ^r 记录 PendingOperation 后可释放运行位, 使 LLM 调用在飞期间其他 Agent 的服务可与其重叠—— $M/M/1$ 抽象在 LLM 等待窗口内失真。
无限缓冲	近似 (未观察到队列容量上限)。

假设	在 ConsistenCy 中的状态
平稳性	存疑（仓库变更突发、负载日周期）。

表 9.1: 排队基线的假设状态表。整体标签：**基线近似**——诚实的记号是“带批化到达的 G/G/c 样系统” [24]；本节任何数值都不构成架构不变式。

9.3 LLM 服务时间为何必须非指数建模

三个来自 LLM 服务系统的实证事实使得“把远程推理当 M 服务”不可辩护：(i) Orca 的迭代级调度与连续组批使一个请求的逗留时间取决于同批负载 [29]；(ii) Page-dAttention 表明服务器端内存管理可同延迟下改变 2-4 倍吞吐——有效服务率 μ 对客户端是隐变量 [30]；(iii) 本报告文献核验中确认的进一步证据（KV 缓存复用使服务时间历史相关）强化同一结论。这些事实不否定 Little 定律（它对任意纪律/分布成立），但否定了把 Erlang-C 当作预测器使用；它们正是 Kingman 变异性项（定理 9.4）值得引用的原因：预算式反压（reserve 阶段）在接近容量之前介入，是对“ μ 不可知”的正确工程响应，而不是速率限制器的缺席。

9.4 CMDP 研究模型（Proposed, 未实现）

把“在 token/成本/截止期约束下调度多审查 Agent”表述为约束马尔可夫决策过程 [31]：CMDP = $(\mathcal{S}, \mathcal{A}, P, c, \{g_k\}, \gamma)$ ，状态含队列构成、剩余预算、提供商可用性、运行计数；动作是准入决定（准入/推迟/取消）； P 在实践中未知（需估计，且 §9.3 的违背使估计困难）；目标

$$\min_{\pi} \mathbb{E}_{\pi} \left[\sum_t \gamma^t c(s_t, a_t) \right] \quad \text{s.t.} \quad \mathbb{E}_{\pi} \left[\sum_t \gamma^t g_k(s_t, a_t) \right] \leq B_k \quad \forall k, \quad (9.5)$$

在有限状态/紧致动作集与 Slater 型严格可行条件下强对偶成立，鞍点处存在平稳随机最优策略 [31]。

强制诚实（三条）。 (i) 当前调度器不是优化器：3 路确定性可行启发式，无值函数、无前瞻。(ii) 预算不在准入处执法：调度器源码零预算引用；ACB 的 token/成本/墙钟预算策略元数据，未被任何路径强制；唯一强制的预算系统是调用授权处的两阶段 reserve/commit (χ_9)。若要忠实建模，约束必须迁到授权边界或扩展准入状态。(iii) costUsdMicros 通道端到端存在但 LLM 门面只报告 token——成本约束在实践中不可消耗。这三条是现状的工程不变式；CMDP 是把“应该是什么”形式化的提议（第二十章研究问题 RQ2）。

第十章 仓库监控与事件触发执行

10.1 监督子系统的真实形状（轮询，而非 Webhook）

仓库监督是轮询模型（源码核验，`heartbeat/repositorySupervisor.ts`）：每仓库 `setInterval(pollIntervalMs)` 驱动一次 `scanRuntime→probe.observe()`（本地 Git 探针）；重叠扫描被生命周期令牌丢弃。每次观测计算观测摘要

$$\text{digest}_t = \text{SHA256}([headSha \parallel \text{UNBORN}, \text{workflowDigest}, \text{normalizedChangedFiles}]), \quad (10.1)$$

摘要不变 $\Rightarrow$ 无待处理变更；变化则武装一个 `debounceMs` 计时器，静默后冲刷（flush）一个去重的 `RepositoryEvent`，其身份 `dedupeKey` 为常量前缀加 `SHA256[repoId, head, workflowDigest]`。工作流摘要覆盖启用的自动化与 `on_change` 绑定：绑定集变化会重新武装事件（摘要变 $\Rightarrow$ 新键），而未启用绑定的仓库保持字节相同摘要（无“事件风暴”的兼容性设计）。失败诚实降级（`degraded` 状态 + 降级脉冲）。监督只观测与记录，不自行入队；审计执行桥同样经规范化路径（`launchDefinitionRun`）以围栏式 `created→queued` 状态守卫单次尝试排空。

10.2 事件触发 vs 周期触发：结构性类比（Strong）

该机制的形态是“Riemann 采样观察下的 Lebesgue 事件”：轮询 + 摘要比较是周期观测者；摘要变化是事件；防抖是驻留时间（`dwell time` / 最小事件间隔）。事件触发采样优于周期采样的经典结论 [32] 在结构上支持“变更触发评审在同等最坏时延与动作预算下浪费更少运行”这一定性主张；Tabuada 的最小事件间隔时间（`minimum inter-event time`）形式化了防抖作为防抖振（Zeno 行为）手段的角色 [33]。但两文的定量定理都以随机被控对象与噪声模型为前提——Git 仓库在提交之间没有动力学：它是一个被无噪声观测的分段常数信号。因此：结构映射 **Strong**；最优性内容 **Weak**。任何“变更触发严格优于 `cron` 触发 $x\%$ ”的定量主张在本报告的框架内都是未证明的（第 20 章研究问题 RQ5）。

10.3 CUSUM/变化检测：不适用（负结果，N/A）

顺序变化检测理论（Page 的连续检查方案 [34] 及其后续）存在的理由是噪声观测下的检测延迟/虚警权衡。摘要比较是无噪声、内容寻址的哈希等值测试：虚警概率 $\approx$ 碰撞概率 ≈ 0 ，漏检概率 $= 0$ ，因此没有阈值可优化、没有权衡可言——它只在与 $a \neq b$ 相同的意义上是“变化检测”。把防抖称作 CUSUM 阈值正是本研究要防止的“数学优雅强迫映射”。变化检测理论在观测变为有噪声时才变得相关：远程镜像的同步滞后、部分写入、基于 mtime 的启发式、CI 指标的漂移检测——这些是假设性的未来通道，记录为条件性映射而非现状描述。

10.4 单运行去重与“缓冲大小为 1”

监督的去重 + 触发计划的“每 (repository, definition, event) 至多一次运行”共同实现了每键缓冲大小为 1、新到替换旧的策略：突发的连续提交不会排队成一串过时的评审运行，而是被合并为最新状态的至多一个待执行计划。信息年龄文献的奠基性洞察——对新鲜度而言“小缓冲 + 丢弃过时更新”优于“无界排队” [35]——正是该设计的理论理由（这是 AoI 理论向本系统真正迁移的一条；其余部分的诚实评估见第十二章）。

第十一章 证据、溯源与可复现性

11.1 证据模型

证据记录为强类型 11 元组：

$$e = (\text{source}, \text{ruleId}, \text{loc}(\text{path}, \text{startLine}, \text{endLine}), \\ \text{conf} \in [0, 1], \text{payload}, \text{prov}), \quad (11.1)$$

$$\text{prov} = (\text{repository}, \text{sha}, \text{analyzer}, \text{analyzerVersion}), \quad (11.2)$$

$\text{source} \in \{\text{ast}, \text{sast}, \text{git}, \text{lint}, \text{symbol}, \text{test}, \text{agent}\}$ 。身份函数为

$$\text{fp}(e) = \text{SHA-256}(\text{canonical}(N(e))) \in \{0, 1\}^{256}, \quad (11.3)$$

其中 N 是 store 侧的确定性规范化（路径规范化——与仓库系统调用同一 `normaliseResourcePath`，拒绝绝对路径/穿越/NUL；store 自行计算指纹，从不信任分析器提交的指纹；校验置信度/行号/溯源；深冻结），`canonical` 是 11 元组的规范 JSON 序列化（递归键排序、数组保序、深度上限 64、拒绝非纯对象与环）。两个设计点值得强调：溯源参与哈希（同一规则在不同修订版本永不共享身份）；身份与标签分离（`evid_⟨uuid⟩` 是非身份标签）。

11.2 理论映射

- **why/where 溯源**：Buneman–Khanna–Tan 的经典区分——“结果由哪些源数据贡献（why）”与“值从哪个源位置拷贝（where）” [36]——与“发现必须引用贡献证据（why）+ 证据钉定（path, 行区间）（where）”一一对应。映射强度：STRONG（形式场景是只读关系查询，非仓库分析，但两个溯源问题逐字迁移）。
- **PROV 词汇**：prov 四元组是 W3C PROV-DM 实体/活动/代理/派生词汇的实例化——证据（实体）由分析器运行（活动）使用快照（实体）生成 [37]。DIRECT（结构层）。

- **内容寻址承诺：**SHA-256-规范元组指纹是 Merkle 哈希树的单叶退化情形 [38]；RepositorySnapshot 的对象库读取同属内容寻址谱系。DIRECT（机制层）。
- **审计日志：**仅追加、全决策入账、永不出现原始句柄的 AuditJournal 站在安全审计日志 [39] 与防篡改日志数据结构（history tree 及其成员/一致性证明）[40] 的谱系上。STRONG（本系统对手模型是内部一致性而非事后取证妥协，声明更弱；一致性证明列为未来升级路径）。
- **供应链证明：**分析器（步骤）绑定输入（钉定快照 = 材料）到输出（证据 = 产品）、步骤身份 = 分析器 + 版本的形态与 in-toto 的 layout+link 元数据同构 [15]。STRONG。
- **模型输出的事后接地：**评审 Agent 的输出在被确定性接地检查（接受/降级/拒绝）之前是不可信的——这是 RARR 式“检索证据并修订输出直到被支撑”[41]的内核化版本：接地依据不是另一个模型，而是第 四 章治理下的确定性证据库。实证动机：原始模型审查输出的行动率类型依赖且不可靠 [4]。

11.3 接地不变式（两级严格度，如实）

命题 11.1 (证据接地). (a) workflow 运行时层面: *schema* 强制 $\forall f \exists E_f \subseteq E_t, E_f \neq \emptyset$ (*evidenceIds* 数组 *min*(1)). (b) 评审工作负载的接地函数 $\text{ground} : \text{Finding}^* \rightarrow \{\text{accept}, \text{downgrade}, \text{reject}\}^*$ 是确定性分类器: 模型引用未知证据 *id* $\Rightarrow$ 拒绝; 未变更文件引用 $\Rightarrow$ 拒绝; 越界行号 $\Rightarrow$ 拒绝; *confirmed* 主张落在变更 *hunk* 之外或无确定性信号 $\Rightarrow$ 降级为 *likely*; 被接受的发现被确定性附加运行证据。 (c) 两级严格度: 遗留评审 *schema* 层 (迁移期) *evidenceIds* 可选——*finding* 可以在 *schema* 层无证据存在; 仅 workflow 运行时层强制 ≥ 1 。

[(a)(b) ENGINEERING_INVARIANT; (c) 记录为限制]

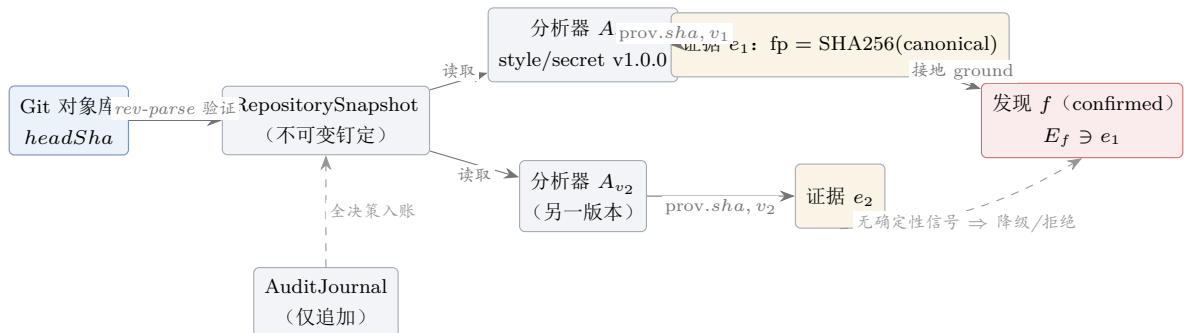


图 11.1： 证据溯源图。快照（蓝）由对象库钉定；分析器带版本读入；证据身份含 (*sha, analyzer, version*)；发现经确定性接地分类器 *ground* 引用证据（红）。虚线标注被 *ground* 排除的路径。

11.4 确定性与指纹可复现性

假设 A6 (分析器确定性 (版本钉定)). A_v 是钉定版本 v 上的全函数、确定性 (字节稳定输出; 全性经孤立代理对替换为 U+FFFD 维持; 输出按 (path, startLine, endLine, ruleId) 确定排序) ——代码可证。

假设 A7 (规范序列化的确定性). JSON 数字的渲染平台稳定 (IEEE-754 双精度在 TS 引擎间的确定性渲染) ——工程假设。

假设 A8 (规范化的单射性). $\text{canonical} \circ N$ 在值域上单射 (不同证据内容产生不同规范串)。**已知边界:** 深度上限 64 的截断与数字渲染合并是确定性碰撞通道, 位于随机预言机模型之外——本假设把它们声明为工程上可忽略, 而非数学上不存在。

假设 A9 (版本可判定). “相同 v ” 可从记录本身判定 (analyzerVersion 在 prov 内)。

命题 11.2 (确定性). 在 A6–A8 下, $(x, v) = (x', v') \Rightarrow \text{fp}(N(A_v(x))) = \text{fp}(N(A_v(x')))$ 。

证明. A_v 确定性 $\Rightarrow$ 结构化输出相等; $N, \text{canonical}, \text{SHA256}$ 是函数 $\Rightarrow$ 哈希相等 (函数复合保持相等; 不涉及概率)。□

[PROPOSITION (A6 代码可证; A7/A7' 工程假设)]

命题 11.3 (碰撞界 (随机预言机理想化)). 把 SHA-256 建模为随机预言机 (显式声明: SHA-256 是固定公开函数, RO 是仅为得出界而用的建模虚构)。设 n 条证据记录的规范编码两两不同 (相同元组的重复是去重而非碰撞), $b = 256$, 则

$$\Pr[\exists i < j : \text{fp}_i = \text{fp}_j] \leq \binom{n}{2} 2^{-b} \leq \frac{n^2}{2^{257}}. \quad (11.4)$$

证明. 逐对使用 union bound; RO 模型下每个不同输入对的碰撞概率为 2^{-256} 。注意 union bound 不需要碰撞事件间的独立性——只需要逐对界。数值: $n = 10^9 \approx 2^{30}$ 时上界 $\leq 2^{-197}$ 。□

[THEOREM_FROM_KNOWN_MODEL (RO 理想化下的 union/birthday 界; Merkle 谱系 [38])]]

11.5 真相不变式 T1/T2 与可复现性 R1

命题 11.4 (真相不变式). **T1** (Git 事实来自 Git 对象库): 快照读取是纯函数 $\text{Read} : (\text{repo}, \text{sha}, \text{path}) \rightarrow \text{blob}$, 经失败关闭的 $\text{rev-parse --verify}$ 钉定; 后续工作树变更不能改变既有快照的观测 (不可变性来自对象库寻址本身)。**T2** (PR 事实来自提供

商 *API*): 外部 *PR* 状态只经提供商中介、失败关闭的路径进入 (未配置 $\Rightarrow$ 类型化错误); 密钥仅服务端; 发布只经授权意图。

[*ENGINEERING_INVARIANT* (*T1* 逐行代码可证; *T2* 部分代码接地、部分模型级补全; 提供商诚实性在模型之外)]

命题 11.5 (可复现性 R1). 钉定快照 + 确定性分析器 $\Rightarrow$ 相等输入产生相等证据指纹 (命题 11.2); 碰撞概率可忽略 (命题 11.3, *RO* 理想化 + *A7*)。这是可复现构建 (*reproducible-builds*) 原则在分析侧的实例化: 同一源、同一过程版本 $\Rightarrow$ 位相同输出。

[*PROPOSITION* (输入侧 *ENGINEERING_INVARIANT*)]

两个风险世界的如实声明。 当前存在两个风险评分世界: 遗留 Python 引擎的 *compose_file_risk* (信号分数的加权钳制: $0.28\text{style} + 0.39\text{structural} + 0.33\text{semantic} + 0.05\text{dup} + 0.5\text{security}$, 带安全地板) 与内核证据置信度——前者的权重不是从后者推导的。任何“风险评分有理论保证”的说法都不成立; 其公理化 (单调性、校准) 是开放问题 (第 20 章研究问题)。

第十二章 仓库与证据新鲜度

本章是“怀疑式文献审计”的产物：我们专门评估了信息年龄（Age of Information, AoI）理论是否适合描述 ConsistenCy 的仓库审查新鲜度，结论是不适用（有窄例外）——并给出更适合的度量。该结论与正面结果同等重要。

12.1 为什么线性时间年龄是错误的原始度量（负结果）

AoI 定义时间年龄 $\Delta(t) = t - u(t)$ （ $u(t)$ 为最近更新时刻）[42]。休眠反例：仓库 90 天无提交且分析钉定在 HEAD—— $\Delta = 90$ 天，而版本滞后 $D = 0$ ，每条发现都完美新鲜。反过来：分析后一小时内涌入 50 个琐碎提交—— $\Delta \leq 1$ 小时而 $D = 50$ ，发现可能引用已不存在的代码。在低变更率区间，时间年龄与真实新鲜度负相关；在高变更率区间它没有信息量。结论：拒绝 $\Delta(t)$ 作为新鲜度度量；它只保留“调度时延描述符”的身份。

12.2 版本滞后（Proposed 度量，VAoI 接地）

定义 12.1 (版本滞后). $D(t) = d(\text{analyzedSha}(t), \text{HEAD}(R_t))$, $d(s_1, s_2) = |\text{anc}(s_2) \setminus \text{anc}(s_1)|$ （从 HEAD 可达而不从被分析提交可达的提交数）。

两个精确警示：(i) d 是非对称拟度量——不满足三角不等式，含义是“HEAD 有而分析没有的提交”； D 小不排除“孤儿分支分析”（分析包含 HEAD 已不包含的提交——“引用不存在的代码”失败模式只被部分捕捉）；分叉历史上 d 的解释需要合并基约定（线性历史上即提交计数）。(ii) 监督的“版本”实际是摘要等价类 $[\text{headSha}, \text{workflowDigest}, \text{changedFiles}]$ ——重配置同 sha 也是新版本； D 是它在线性历史上的影子。

该度量的形状并非新造：版本信息年龄（Version AoI）——监控者落后源的版本计数——正由 Buyukates 等人的工作线建立 [43]。映射判定：定义层 USEFUL（已有文献）；把 VAoI 实例化到“评审证据监督于 Git DAG 之上” = PROPOSED 扩展（Gossip

网络分析机件不适用)。

12.3 检测时延与新鲜证据时刻（初等界）

假设 A10 (新鲜度界的确定性前提). 变更时刻外生；扫描不重叠 ($\delta_{\text{scan}} \leq T_{\text{poll}}$ ；重叠扫描被丢弃)；触发变更之后不再发生摘要变化（无再武装链）。

命题 12.2 (新鲜证据时刻的上界). A9 下，时刻 θ 的变更最迟在 $\theta + T_{\text{poll}} + \delta_{\text{scan}}$ 被检测（下一个轮询栅格点 + 扫描时长；摘要比较无噪声——这正是第 10 章 CUSUM 不适用判定的事实基础 [34]），再经 T_{deb} 后被冲刷（防抖恰在最后一个变更后 T_{deb} 冲刷——A9 使其精确），再经 T_{run} 后有终态运行（命题 8.3 的 F1–F3）。故对固定时刻 t ，尾随窗口 $W := T_{\text{poll}} + \delta_{\text{scan}} + T_{\text{deb}} + T_{\text{run}}$ 之外的滞后不可能存在：

$$D(t) \leq N((t - W, t]) \quad (\text{提交计数过程}) \quad (12.1)$$

证明. 逐段复合三个时延不等式 + 命题 8.3；窗口包含关系来自“变更至多等待 W 即被终态运行覆盖”。□

[PROPOSITION (A9 下精确；A9 违背——再武装链、扫描重叠、崩溃（单次尝试 $\Rightarrow$ 时延无界）——即其失败条件)]

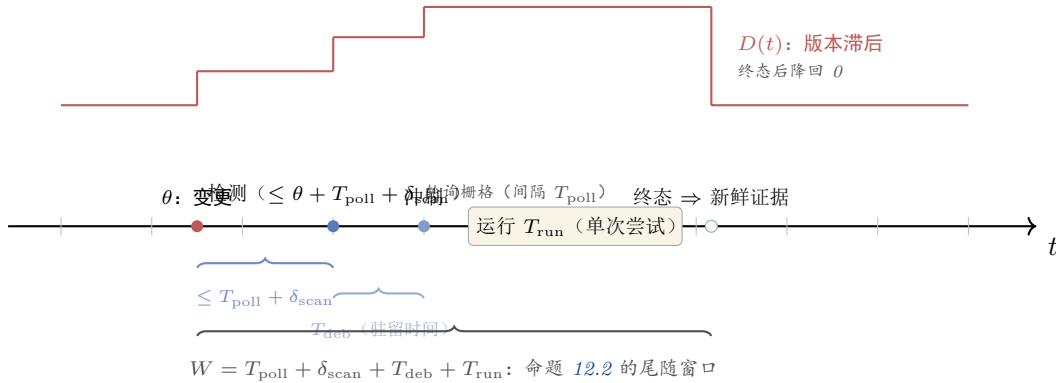


图 12.1: 证据新鲜度时间线（理论示意图）。变更 θ 经检测（轮询栅格 + 扫描）、防抖（驻留时间）、运行后产生新鲜证据； $D(t)$ 在此期间阶梯上升、终态后降回。所有量为理论参数，非实测。

12.4 平均值与低速率近似（Proposed 度量/推导）

假设 A11 (泊松变更（随机化前提）). 变更时刻是速率 λ_c 的泊松过程，且与轮询栅格独立。

在 A10 下，变更的栅格相位在 $[0, T_{\text{poll}})$ 上均匀（泊松增量平稳 + 与栅格独立；无检查悖论），故 $\mathbb{E}[\text{等到下一轮询}] = T_{\text{poll}}/2$ 。冲刷参照的平均检测-冲刷时延为

$$\mathbb{E}[\text{delay}_{\text{flush}}] = \frac{T_{\text{poll}}}{2} + \delta_{\text{scan}} + T_{\text{deb}}, \quad (12.2)$$

（检测参照则为 $T_{\text{poll}}/2 + \delta_{\text{scan}}$ ；两处都必须显式选择参照——本报告统一采用冲刷参照）。由更新报酬定理 [44]，低速率时长期平均等于逐变更均值。

固定时刻的期望滞后（A9 与 A10 的交界，如实表述）。 由命题 12.2 与泊松平稳增量：**A9 成立时**（确定性世界）， $\mathbb{E}[D(t)] \leq \lambda_c W$ 在固定 t 处精确。在 A10 下，A9 的“无后续变更”以概率 $1 - e^{-\lambda_c T_{\text{deb}}} > 0$ 被违背——再武装链使尾随窗口包含关系失守，正确的低速率近似是

$$\mathbb{E}[D(t)] \lesssim \lambda_c (W + \mathbb{E}[\text{再武装扩散}]), \quad \text{当 } \lambda_c (T_{\text{poll}} + T_{\text{deb}}) \ll 1 \text{ 时误差为 } O(\lambda_c (T_{\text{poll}} + T_{\text{deb}})), \quad (12.3)$$

再武装暴露窗口延伸到下一个扫描时刻（故条件用 $T_{\text{poll}} + T_{\text{deb}}$ 而非仅 T_{deb} ）。**明确不主张“峰值滞后”的期望界：**无界时域上窗口计数的运行最大值几乎必然无界，任何此类主张在 A10 下都是假的；峰值陈述只允许在声明的有限时域 H 上给出。

12.5 AoI 迁移的最终账目

AoI 组成部分	判定	理由
线性/峰值时间年龄	N/A	休眠反例；版本离散状态。
缓冲-大小-1 洞察	USEFUL	“单运行去重 + 新到替换”的真实理由 [35]。
峰值年龄记账	USEFUL	检测时延 + 运行时间的初等界（命题 12.2）。
版本 AoI	USEFUL/PROPOSED	$D(t)$ 的定义接地 [43]；Git-DAG/摘要等价类两个皱褶是真正的扩展。
事件触发 vs 周期	STRONG(结构)	驻留时间框架 [33]；无定理迁移（无随机被控对象） [32]。
更新或等待最优采样	WEAK	观测免费且精确时退化为“总是更新”；只有给运行定价后才非平凡。
信道/队列分析机件	N/A	无物理信道、无争用；拉取观测给出确定性界。
CUSUM/变化检测	N/A	哈希等值无虚警/延迟权衡 [34]。

AoI 组成部分	判定	理由
----------	----	----

表 12.1: AoI/新鲜度理论向 ConsistenCy 的迁移账目（怀疑式审计结论）。

第十三章 ConsistenCy 统一数学模型

13.1 统一模型（Proposed：透镜，非实现）

把前述各部件装配为一个离散时间演化系统：

$$X_{t+1} = \Phi(X_t, u_t, \zeta_t), \quad u_t = \pi(X_t), \quad (13.1)$$

其中 $X_t = (R_t, S_t, Q_t, \mathfrak{A}_t, C_t, V_t, K_t, E_t, P_t, B_t)$ 为全局状态（第三章）， u_t 为准入/调度控制， ζ_t 为外生噪声（仓库变更、提供商时延/停机、崩溃事件）。**硬约束**（任何目标不得交换）：(i) 授权不变式 S1-S3；(ii) 预算不变式 $used_c + reserved_c \leq max_c$ （两阶段，在授权处执法——注意与“准入处无预算执法”的现状差距，第九章）；(iii) 依赖可用性（节点仅当 DAG 前驱终态时执行）；(iv) 证据有效性（发布发现满足接地， workflow 运行时层）。目标草图：

$$\min_{\pi} \mathbb{E} \left[\sum_t (w_{\text{lat}} \text{latency}_t + w_{\text{stale}} g(D(t)) + w_{\text{cost}} \text{cost}_t + w_{\text{risk}} \text{risk}_t) \right] \quad \text{s.t. S1-S3 与资源约束}, \quad (13.2)$$

其中 g 为未校准的滞后代价函数、风险聚合的公理未建立（第二十章）。

必须显式声明：该统一模型是一个形式透镜。实现的 π 是 3 路确定性可行启发式（非优化器）；没有任何组件计算 Φ 或上述目标。它的价值是把“调度/新鲜度/成本/风险之间的权衡”从口头讨论提升为有明确自由度的数学问题，并为第二十章的研究问题提供共享记号。

非统一性自白（non-unification confession）。 复查全报告的命题可以发现：没有任何一个命题同时联合使用多个状态分量——S1/S2 只读 (C_t, B_t) ；正交性命题只对照 (C_t) 与 (K_t, V_t) ；排队节只读 Q_t ；新鲜度节只读 R_t 与 E_t 的时戳；证据命题只读 E_t, S_t 。因此本报告所称的“统一模型”应更准确地读作**共享记号（shared notation）**：一个让各章局部命题可以无歧义互相引用的公共状态空间，而不是一个产生了跨分量联合定理的模型。真正的联合优化（例如同时约束 Q_t 与 $D(t)$ 的策略类）是第二十章 RQ1/RQ2 的待做内容，不是本报告已完成的内容。我们把这一点写在明处，以对抗“把章节摘要符号化后称作统一模型”的过度包装。

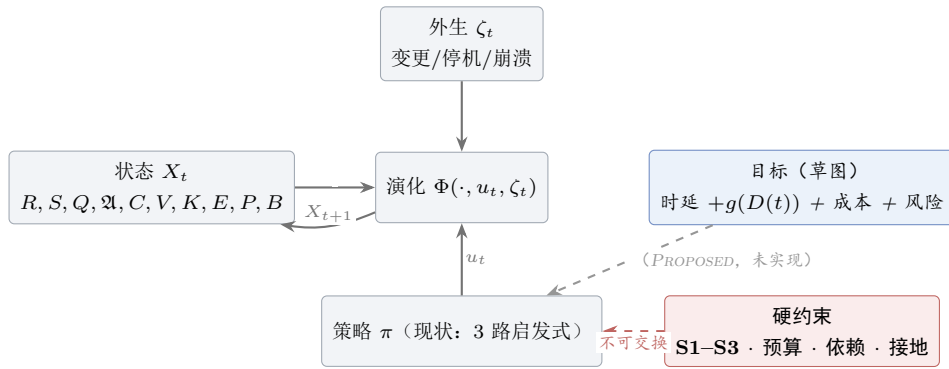


图 13.1: 统一形式模型 (PROPOSED_CONSISTENCY_FORMAL_MODEL)。红色为任何策略都不得违背的硬约束；蓝色目标仅是研究草图——实现的 π 不优化它。

第十四章 形式化安全/活性性质清单

下表汇总全报告的性质，含假设与“已执法 vs 期望性”的区分：

ID	性质	形式内容	关键假设	状态
S1	无授权则无受保护效果	$\Box(\text{Exec} \Rightarrow \text{Auth}(m(q))=1)$	A1–A3; 边界: 网 关穿透依赖 broker 拒绝 (完 全中介者是 broker)	工程不变式 (调用路径代 码可证; 旁 路不存在性 是假设)
S2	撤销支配生命周期传播	中介点 $\succeq t_r$ 裁决恒 0, 与 K_t 无关; intent carve-out	A1–A5	命题 (A4/A5 代 码可证)
S3	数据面不能直接改变裁决	裁决面独立性 (单步、 固定状态, 命题 8.2)	数据 $\rightarrow$ 权威的唯 一通道是内核侧 Ring 校验发放	命题
T1	Git 事实来自对象库	Read : $(repo, sha, path) \rightarrow$ $blob$ 纯函数、失败关 闭钉定	对象库完整性 (模型外)	工程不变式
T2	PR 事实来自提供商 API	提供商中介、失败关 闭、密钥服务端、发 布经意图	提供商诚实性 (模型外)	工程不变式 (部分代码接 地)
E1	发现引用持久证据	工作流运行时层 $\forall f \exists E_f \neq \emptyset$; 确定性 ground	schema $\text{min}(1)$; 差距: 遗留层可选	工程不变式 (仅工作流 层)
R1	可复现证据指纹	命题 11.2+11.3	A6–A8 + A7'; RO 理想化	命题 (输入 侧工程不变 式)

ID	性质	形式内容	关键假设	状态
L1	准入的合格 Agent 最终推进	在公平派发、依赖可用、预算未竭、提供商可用、无崩溃之下	F4/F5 为期望性；HOL 饥饿、PENDING 吸收、 χ_9 拒绝、单次尝试均是其反例条件	命题（假设下）；截止期执法是唯一已执法的活性邻接机制（诚实失败而非进展）

表 14.1: 形式化性质总表。S = Safety, T = Truthfulness, E = Evidence, R = Reproducibility, L = Liveness。

L1 的反例条件值得展开。 “最终推进”在下列任一情形下失败，且每一种都设计上被接受为诚实失败而非缺陷修复目标：(i) 高优先级持续饱和使低类饥饿（定理 9.5）——由截止期取消兜底为“取消”这一终态；(ii) 共效从未提供使 Fiber 永久 PENDING；(iii) χ_9 预算拒绝是条件活性的边界（安全优先于活性的直接体现）；(iv) LLM 无重试——一次修复尝试后类型化失败；(v) 崩溃后启动恢复把执行中标记为失败。换言之，ConsistenCy 的活性哲学是“公平假设下的进展 + 假设失守时的诚实终态”，而非尽力掩盖的可用性最大化——与第 5 章指出的 OTP 分歧一致。

第十五章 架构决策的理论解释

下表把九项代表性架构决策映射到其工程理由、理论解释与数学性质，并标注证据强度。这是“工程直觉 $\rightarrow$ 可检查对象”转化的核心交付物之一。

架构决策	工程理由	理论	数学性质	强度
逐 syscall 授权（9 步链重评）	防缓存授权过期、防 TOCTOU、撤销即时生效	引用监视器/完全中介 [8, 11, 16]	S1: $\Box(\text{Exec} \Rightarrow \text{Auth})$	DIRECT
撤销永久单调 + 下一调用拒绝	简化撤销语义；安全不依赖传播时序	撤销经中介（退化情形） [9, 12]	S2: 支配性（命题 4.3）	DIRECT
Coeffect 与授权两次独立检查	可用性与权威分属不同时间尺度	vat 模型/事件循环响应式 [10]	正交性（命题 5.2、5.3）	DIRECT
Context VM 与能力分离	“看见什么”与“能做什么”不同轴	关注点分离/最小特权 [8]	Visible $\not\vdash$ Auth（命题 6.2）	DIRECT
intent 分发门（不可逆效果）	外部副作用需要幂等与显式意图	幂等生产者/效果分级的业务效果 [6]	dispatch(intent) $\Rightarrow$ $\neg$ 内联 (§八)	STRONG
Scheduler 属于 Kernel（非插件）	准入/并发/截止期是权威而非功能	准入控制 + 非抢占 HOL [25]	确定性准入函数；饥饿形状（定理 9.5）	DIRECT（词汇）
Evidence-first findings	模型输出不可信，需事后接地	why/where 溯源 + 事后归因 [36, 41]	ground 分类器；E1	STRONG
RepositorySnapshots SHA 钉定	读取必须是内容函数	内容寻址承诺 [38]	T1; A_v 复合 (R1)	DIRECT

架构决策	工程理由	理论	数学性质	强度
持久触发计划 + 去重键	恰好一次发起、 防事件风暴	AoI 缓冲-1 洞 察 [35]	单飞 CAS 状态 机 (§八)	USEFUL

表 15.1: 架构决策 → 理论解释映射表。

第十六章 与相关系统/论文的比较

系统	权威中介	证据/接地	验证程度	失败哲学
ConsistentCy v3	逐步 syscall 9 能力链； intent 门	确定性分析器 + 内容指纹 + 强制 ground	无形式证明； 模型级命题 + 源码核验	fail-closed、 单次尝试、 诚实终态
seL4 [13]	能力内核，同 步中介	N/A（机制不 做证据）	完整机器检 查函数正确 性证明	正确性由证 明承担
Capsicum [14]	能力模式（沙 箱内禁全局命 名空间）	N/A	无证明；部署 经验	细粒度授权 实践
in-toto [15]	签名 layout 授权步骤	链接元数据绑 定材料 → 产 品	无证明；部署 （供应链）	步骤可验证
CT/history tree [40]	N/A	防篡改仅追加 日志 + 一致 性证明	结构证明	公开可审计
Erlang/OTP [16]	能力模型 （进程隔离）	N/A	无证明；工业 可靠性记录	让它崩溃 + 重启掩盖
ReAct [1]	无（宿主约 定）	无	N/A	提示层约束
AutoGen [2]	无（会话编 程）	无	N/A	应用层
MetaGPT [3]	无（SOP 编 码）	无	N/A	应用层
CodeReviewer [15]	N/A	无证据接地 （模型端到端）	实证（预训 练）	概率输出

系统	权威中介	证据/接地	验证程度	失败哲学
----	------	-------	------	------

表 16.1: 与相关系统/论文的比较。LLM Agent 框架一行显示的“无权威中介”正是 ConsistenCy 的存在理由；与 OTP 的失败哲学分歧（诚实失败 vs 重启掩盖）见第 五 章。

定位声明：ConsistenCy 不是“又一个 Agent 框架”，而是把 OS 内核六十年的权限理论 [7, 8, 11] 与可复现计算的溯源理论 [36–38] 装配到 LLM Agent 工作负载上的基础设施；ReAct/AutoGen/MetaGPT 属于其上可承载的工作负载形态，而非同类竞争者。

第十七章 哪些理论已经支撑当前实现

图 17.1 以矩阵形式总结全部理论映射的强度分布：DIRECT 集中在安全与分离公理（引用监视器、撤销、正交性、内容寻址），STRONG 集中在生命周期/溯源/意图门，USEFUL 在排队词汇与缓冲洞察，PROPOSED 在 CMDP 与版本滞后度量，N/A 记录了 Petri 机件、CUSUM 与 AoI 线性年龄三组负结果。

概念 \ 强度	DIRECT	STRONG	USEFUL	PROPOSED	WEAK	N/A
逐调用授权/引用监视器	•					
撤销支配	•					
可用性/授权正交	•					
可见性/权限正交	•					
内容寻址身份	•					
ACB/Fiber LTS	•					
PROV 溯源结构	•					
Fiber 监督谱系		•				
证据 why/where		•				
意图门/幂等		•				
防篡改日志谱系		•				
事件触发结构		•				
LLM 服务时间实证		•				
排队记号/稳定性/Little			•			
AoI 缓冲-1 洞察			•			
峰值年龄记账			•			
模型检验扩展				•		
CMDP 准入研究模型				•		
版本滞后 $D(t)$				•		
两阶段预算预留					•	
更新或等待最优采样					•	
Petri/WF-net 机件						•
CUSUM/变化检测						•
线性时间年龄						•

图 17.1: 理论支持强度矩阵 (图例: 蓝 =DIRECT, 绿 =STRONG, 琥珀 =USEFUL, 玫瑰 =PROPOSED/N/A, 灰 =WEAK)。逐项论证与文献见第 四-十二 章及附录 B。

第十八章 哪些只是提议研究方向

以下内容不是当前系统的属性，把它们写成现状即构成理论包装。（评估深度声明：本章列举的负结果与新方向中，AoI/新鲜度经受了最深的专项审计（第 十二章），Petri 与 CUSUM 为边界论证（§八、§十），深度不一；此处列出负结果的目的是划定“已确立理论”的边界，而非宣称对三者的评估等量齐观。）

1. **CMDP 准入优化**：现状是 3 路可行启发式；预算不在准入处执法；成本通道不可消耗。
2. **版本滞后度量 $D(t)$ 与触发策略优化**：度量是本报告的提议（VAoI 接地）；“事件触发 vs 周期的定量优劣”是未证明命题。
3. **积 LTS 的模型检验**：48 状态空间是现成的检验对象，但未接线。
4. **统一模型 Φ 与目标函数**：透镜，无组件计算它。
5. **风险评分公理化**：两世界并存的现状下，任何单调性/校准主张都未建立。
6. **审计日志一致性证明**：仅追加接口 + 指纹纪律是现状；Crosby 式密码学防篡改 [40] 是升级路径。
7. **ensemble 可靠性模型**：没有数据支撑独立假设，禁止 $\prod_i p_i$ 式计算；贝叶斯集成仅列为经验研究议程。

第十九章 局限

19.1 模型与方法论的局限

1. **模型级而非实现级证明**。除标注“代码可证”的构造性论据外，S1–S3、正交性与活性命题证明的是架构模型 $\mathcal{M}_C$ 。“不存在旁路”（A1 的全称否定）只有形式化验证才能消除——seL4 正是走了这条路 [13]，而 ConsistentCy 没有。
2. **OS 围堵边界**。除子进程隔离（剥离父环境秘密 + RPC 中介）外，OS 级文件系统/网络围堵未强制；Ring 3 in-process 内建组件与宿主共享进程边界。S1 只约束经中介的效果。
3. **外部库公理**。Fiber LTS 语义由 Cordis 4.0.0-rc.8 库拥有；库升级会重新打开第七章结论。
4. **理想化**。碰撞界依赖随机预言机 + A7'（规范化单射为工程假设）；排队节整体是声明的基线近似（假设表 9.1 含五项违背/存疑）。
5. **诚实差距清单** (docs≠code)：授权链“8 步”注释 vs 9 步实现；预算注释声称调度器消费而源码无引用；ACB 预算元数据未执法；成本通道不可消耗；Coordinator 内存去重；网关未注册动作穿透；遗留 schema 允许无证据 finding。每条都已编入相应命题的边界条件。
6. **文献覆盖**。45 篇经核验文献覆盖五条 lane 的正典；两篇现代平行工作 (CaMeL [6]) 为未经评审预印本，其元数据以 2026-08 为准。

19.2 度量与评估的局限

本报告没有任何实测基准：排队曲线是理论图（图 9.2 图注已声明）； $D(t)$ 、 $\mathbb{E}[\text{delay}_{\text{flush}}]$ 等度量的参数 ($T_{\text{poll}}, T_{\text{deb}}, \dots$) 未在真实部署上标定；风险评分两世界并存。经验评估（真实仓库、真实 LLM、真实 PR 流）是把本报告的透镜变成工程反馈的必要下一步。

第二十章 未来研究议程

开放问题 20.1 (RQ1 高频提交下的证据滞后最小化). 在提交率 λ_c 、轮询 T_{poll} 、防抖 T_{deb} 、运行成本与并发上限约束下, 最小化 $\mathbb{E}[g(D(t))]$ (或有限时域峰值) 的策略类是什么? 变更触发与周期触发的定量 crossover 在哪个参数区域? (PROPOSED; 需要校准的 g .)

开放问题 20.2 (RQ2 token/成本约束下的多 Agent 调度). 把 χ_9 的两阶段预算提升为 CMDP 准入约束 (或证明确定性启发式的竞争比); 估计 μ 的在线方法; $c > 1$ 与批化到达下的重交通界。 (PROPOSED [31].)

开放问题 20.3 (RQ3 撤销与异步生命周期的安全一致性). 陈旧门面窗口 W_{stale} 的有界性证明 (微任务队列调度分析); 意图门 carve-out 的“恰好一次效果发起”在 Sink 持久性故障模型下的形式化。

开放问题 20.4 (RQ4 可验证的 Agent 产出软件证据). 把 ground 从分类器扩展为论证框架 (发现 = 主张, 证据 = 支撑, ground = 攻击关系); 证据集增长下接受精度的单调性; 签名/一致性证明的引入路径 [40]。

开放问题 20.5 (RQ5 事件触发评审的何时更优). 在 g 校准后, 更新报酬框架下变更触发 vs 周期触发的长期平均成本比较; 与“驻留时间”参数 T_{deb} 的联合优化 [32, 33]。

开放问题 20.6 (RQ6 上下文分配作为约束信息分配). Visible 投影在 token 预算下的最优页面集选择 (工作集理论的 LLM 版); 与 Auth 的正交性保持。

开放问题 20.7 (RQ7 Agent Harness 的长期仓库稳定性). 把“多 Agent 持续修改下仓库质量的平稳性”形式化: 风险评分的公理化 (单调、校准聚合)、评审漏检过程的诚实建模 (无独立假设)、以及两风险世界的合并。

结论

本报告对 ConsistenCy v3 完成了一次完整的理论化闭环: 以当前工作树为唯一事实来源, 把 14 个源码核验的组件映射到六个理论域; 经 45 篇网络核验文献的支持强

度分级；构造了覆盖授权、撤销、正交性、生命周期、工作流、排队、证据与新鲜度的形式模型 $\mathcal{M}_C$ ，其中包含 2 个已确立定理的直接实例、11 个模型级命题（含全部假设、证明与失败条件）与 3 组负结果（Petri 机件、CUSUM、线性时间年龄）。敌意数学审计（PASS_AFTER_FIXES，全部修正已并入）与独立引文审计（零虚构，5 处修正已应用）之后，结论可以概括为三句话：

1. **ConsistenCy 的核心安全直觉是理论上的直系后代：**引用监视器、对象能力、撤销经中介与内容寻址承诺在机制层一一对应（DIRECT），其公理“ $\text{Coeffect} \neq \text{Authorization}$ ”“ $\text{Context VM} \neq \text{Permission System}$ ”获得了精确的数学表述——安全只由 Auth 承担，可用性与可见性只影响活性。
2. **最有价值的部分恰恰是诚实的负结果：**AoI 的线性年龄、Petri 网机件与 CUSUM 都被判定为不适用并给出理由；CMDP、版本滞后与模型检验被明确标记为提议而非现状。
3. **从工程直觉到研究议程的通道已经打开：**统一模型 Φ 、七个开放问题与“真实测量”的缺失共同构成下一步——让透镜遇到数据。

附录 A 完整数学推导

本附录给出正文中压缩陈述的完整推导。所有推导针对架构模型 $\mathcal{M}_C$ ；随机性只在显式声明的假设（A9/A10、随机预言机）下出现。

A.1 Erlang-C 公式的推导梗概（定理 9.2）

$M/M/c$ 的稳态是生灭链： $\pi_{n+1} = \frac{\lambda}{\min(n+1, c)\mu} \pi_n$ 。记 $a = A_e = \lambda/\mu$ ，则 $\pi_n = \pi_0 a^n/n!$ ($n \leq c$)， $\pi_n = \pi_0 a^n/(c! c^{n-c})$ ($n > c$)。等待概率即 Erlang-C：

$$P_{\text{wait}} = \sum_{n \geq c} \pi_n = \frac{a^c}{c!} \cdot \frac{1}{1-\rho} \cdot \left[\sum_{k=0}^{c-1} \frac{a^k}{k!} + \frac{a^c}{c!} \cdot \frac{1}{1-\rho} \right]^{-1}, \quad \rho = \frac{a}{c} < 1. \quad (\text{A.1})$$

条件等待是速率为 $c\mu$ 的指数剩余服务 k 次 (k 为到达时队长 $-c$)，几何混合后 $\mathbb{E}[W_q | \text{wait}] = 1/(c\mu - \lambda)$ ，故 $\mathbb{E}[W_q] = P_{\text{wait}}/(c\mu - \lambda)$ 。 $c = 1$ ： $P_{\text{wait}} = \rho$ ， $\mathbb{E}[W_q] = \rho/(\mu - \lambda) = \rho \cdot \tau/(1 - \rho)$ 。到达看到稳态（PASTA）使“到达时队长分布 = 稳态分布”成立——这是把 π 读作到达分布的合法性来源。以上为 $M/M/c$ 理想化内的定理 [25]。

A.2 Little 定律的证明梗概（定理 9.3）

以 $A(t)$ 记 $[0, t]$ 到达数、 $D(t)$ 离去数，系统内人数 $L(t) = A(t) - D(t)$ 。每个顾客的累积在系统时间 $\int_0^t L(s) ds = \sum_{i \text{ 已离去}} W_i + \sum_{i \text{ 在系统}} (\text{部分})$ 。若 (i) 到达率极限 $\lambda = \lim_{t \rightarrow \infty} A(t)/t$ 存在且有限，(ii) 均值逗留 $W = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i \leq n} W_i$ 存在且有限，(iii) 顾客最终离去（无永久滞留），则由 Cauchy-Stolz 型论证 $L = \lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t L = \lambda W$ [26]。证明完全不使用分布假设、服务纪律或优先级——这正是它在表 9.1 的违背下仍然可用的原因；代价是它只约束均值关系，不给出 W 本身。

A.3 非抢占 HOL 优先级均值等待（定理 9.5）

类 k 顾客的等待 = 残余服务 R （到达时刻正在服务的剩余时间）+ 到达时已在队的类 $\leq k$ 工作量 $W_{\text{backlog}}^{(k)}$ + 等待期间新到的类 $< k$ 工作量 $W_{\text{new}}^{(k)}$ 。均值版本（指数平滑假设下的标准分解）：

$$\mathbb{E}[W_q^{(k)}] = \mathbb{E}[R] + \sum_{i \leq k} \lambda_i \mathbb{E}[W_q^{(i)}] \mathbb{E}[S_i] + \sum_{i < k} \lambda_i \mathbb{E}[W_q^{(k)}] \mathbb{E}[S_i], \quad (\text{A.2})$$

其中 $\mathbb{E}[R] = W_0 = \sum_i \lambda_i \mathbb{E}[S_i^2]/2$ （平均剩余服务悖论）。对 $k = 1, 2, \dots$ 递推求解得

$$\mathbb{E}[W_q^{(k)}] = \frac{W_0}{(1 - R_{k-1})(1 - R_k)}, \quad R_k = \sum_{i \leq k} \rho_i \quad (\text{A.3})$$

（ R_k 为类 $\leq k$ 累积利用率 [25]。）分母结构即饥饿形状： $R_{k-1} \uparrow 1$ 时类 k 均值等待 $\sim (1 - R_{k-1})^{-1}$ 爆炸，即使 $R_{\text{all}} = \rho < 1$ 。P3 的“严格优先级 + 类内 FIFO”是其在两类（高/低）情形的实例；ConsistenCy 的兜底是 P1 截止期取消（把无限等待转化为终态），不是公平份额 [28]。

A.4 碰撞界的完整证明（命题 11.3）

设规范编码 $\{c_1, \dots, c_n\}$ 两两不同（A7' + 前置条件）， $\text{fp}_i = H(c_i)$ ， H 为随机预言机。对固定对 (i, j) ： $\Pr[H(c_i) = H(c_j)] = 2^{-256}$ （RO 在不同输入上取值独立均匀）。则

$$\Pr\left[\bigcup_{i < j} \{\text{fp}_i = \text{fp}_j\}\right] \stackrel{\text{Boole}}{\leq} \sum_{i < j} \Pr[\text{fp}_i = \text{fp}_j] = \binom{n}{2} 2^{-256} \leq \frac{n^2}{2} 2^{-256} = \frac{n^2}{2^{257}}. \quad (\text{A.4})$$

关键点：Boole 不等式对事件独立性零假设——它只需要逐对概率界，而逐对界来自 RO 在两个不同输入上的独立取值。因此本证明没有走私任何未声明的独立性假设。数值代入 $n = 2^{30}$ ：上界 $= 2^{60}/2^{257} = 2^{-197}$ 。

A.5 确定性命题（命题 11.2）

设 $(x, v) = (x', v')$ 。A6： A_v 是函数且 $A_v(x) = A_v(x')$ （结构化输出相等——注意这要求 A6 的字节稳定版本而非仅仅“语义确定”）。 N （路径规范化、校验、冻结）是函数： $N(A_v(x)) = N(A_v(x'))$ 。canonical（A7 确定性渲染 + A7' 单射，这里只需确定性）是函数。SHA-256 是函数。函数复合保持相等： $\text{fp}(N(A_v(x))) = \text{fp}(N(A_v(x')))$ 。■。注意该证明对“同输入同输出”零概率假设；全部重量压在 A6/A7 的工程事实上，A7' 只在逆命题（不同内容 $\Rightarrow$ 不同指纹）中需要。

A.6 新鲜证据时刻上界的完整证明（命题 12.2）

设变更（摘要变化）发生在 $\theta \in (t_g, t_{g+1}]$, t_g 为轮询栅格点，栅格间隔 T_{poll} 。(1) 检测：下一次观测在 t_{g+1} 开始，扫描时长 δ_{scan} , A9 保证不重叠 ($\delta_{\text{scan}} \leq T_{\text{poll}}$) 且该次观测必看到新摘要（摘要只依赖仓库状态，观测无噪声）；故检测完成时刻 $\leq t_{g+1} + \delta_{\text{scan}} \leq \theta + T_{\text{poll}} + \delta_{\text{scan}}$ 。(2) 冲刷：防抖计时器在检测时刻武装，A9（无后续变更）下恰经 T_{deb} 冲刷；冲刷时刻 $\leq \theta + T_{\text{poll}} + \delta_{\text{scan}} + T_{\text{deb}}$ 。(3) 终态：冲刷触发计划（去重键唯一 $\Rightarrow$ 发起一次运行），运行按命题 8.3 的 F1–F3 在有限步内到达终态；以 T_{run} 记其上界，新鲜证据时刻 $\leq \theta + W$, $W := T_{\text{poll}} + \delta_{\text{scan}} + T_{\text{deb}} + T_{\text{run}}$ 。(4) 窗口包含：任意固定 t , 若某提交 $u \in (t - W, t]$ 且其触发链未完成，则与 (1)–(3) 矛盾；故未被终态运行覆盖的提交只能落在 $t - W$ 之前，即 $D(t) \leq N((t - W, t])$ 。失败条件即 A9 的三违背：再武装链 ($D(t)$ 可超窗)、扫描重叠（有效栅格变宽）、崩溃（单次尝试 $\Rightarrow$ 时延无界）。

A.7 栅格相位均匀性与平均冲刷时延

A10 下, $[0, T_{\text{poll}})$ 上的相位分布：泊松过程与独立栅格的平移不变性给出 $P(\text{phase} \in ds) = ds/T_{\text{poll}}$ （平稳增量 + 独立性；无检查悖论——栅格不依赖变更过程）。冲刷参照：

$$\mathbb{E}[\text{delay}_{\text{flush}}] = \mathbb{E}[\text{phase 的剩余等待}] + \delta_{\text{scan}} + T_{\text{deb}} = \frac{T_{\text{poll}}}{2} + \delta_{\text{scan}} + T_{\text{deb}}, \quad (\text{A.5})$$

均匀分布的期望剩余 = $T_{\text{poll}}/2$ 。更新报酬定理 [44]：长期平均冲刷时延 = 每周期望总延迟/每周期望变更数；低速率 ($\lambda_c(T_{\text{poll}} + T_{\text{deb}}) \ll 1$, 再武装概率 $1 - e^{-\lambda_c T_{\text{deb}}} \approx \lambda_c T_{\text{deb}}$ 可忽略) 时逐变更均值即长期平均。交界警示（正文 §十二）：固定 t 的 $\mathbb{E}[D(t)] \leq \lambda_c W$ 只在 A9（确定性世界）内精确；A10 世界里必须加再武装扩散修正项，且无界时域“峰值”主张非法。

A.8 撤销支配与传播无关性（命题 4.3、5.3）

见正文证明；此处补全独立性论证的语义细节。Auth 的定义（定义 4.1）是显式给出的公式，其自由变元集 $\text{FV}(\text{Auth}) = \{C_t, B_t, h, a, \alpha, r, s, t\}$ 由构造确定。设 $\mathcal{G}_t = (K_t, V_t, P_t, \mathfrak{A}_t, Q_t)$ 为任意其他分量组。对任意两个轨迹片段 (X, X') 满足 $C = B$ （逐分量）而 $\mathcal{G} \neq \mathcal{G}'$ ：Auth 在两轨迹上逐点相等——因为求值只读取 FV。故“ δ_{prop} 任意（含 ∞ ）”与“窗口内全部拒绝”不矛盾：前者只改变 $\mathcal{G}$ （Fiber/K 侧），后者由 C 侧单调决定。■

A.9 积 LTS 良定义性（命题 7.2）

$\varphi : L_{\text{sched}} \rightarrow L_{\text{fiber}}$ 部分函数且两标签集不相交。定义积迁移 $(s, f) \xrightarrow{\ell} (s', f')$ 当且仅当 (a) $\ell \in L_{\text{sched}}$ 、 $s \xrightarrow{\ell} s'$ 、 $f = f'$ ，且 $\varphi(\ell)$ 未定义或与 f 的共存约束一致（fiber 同步执行 $\varphi(\ell)$ ）；或 (b) $\ell \in L_{\text{fiber}} \setminus \varphi(L_{\text{sched}})$ （如自主 dispose）、 $f \xrightarrow{\ell} f'$ 、 $s = s'$ 。每个复合标签下的迁移由分支条件互斥给出唯一性—— φ 的函数性排除“一个调度标签对应两个 fiber 标签”的歧义。不可达性：(NEW, ACTIVE) 要求存在使 fiber 激活的标签共现，而 NEW 下唯一使能标签是 register（ φ 未定义于其上），矛盾；(SUSPENDED, ACTIVE) 同理由 suspend 的 φ -像 cleanup 排除。■

附录 B 文献映射矩阵

下表为“ConsistenCy 概念 × 理论 × 关键文献 × 支持强度”的完整矩阵（正文的浓缩版是图 17.1 与表 15.1）。文献键与 references.bib 一致；全部 45 篇均经 2026-08-28 网络核验（研究笔记 02/06 记录了核验证据与五处修正）。

ConsistenCy 概念	理论	关键文献	强度	备注
默认拒绝 broker、9 步链、逐调用重评	引用监视器/完全中介/失败默认	anderson1972; saltzer1975	DIRECT	RVM 三性质逐条对上；broker 是完全中介者（A1 边界）
逐调用重评（不缓存）+ sha 钉定作用域	TOCTOU	bishop1996	DIRECT	定义了所防御的漏洞类
不可猜测 256 位句柄；12 位指纹审计能力模式（默认拒绝运行时）	对象能力不可伪造引用 OS 能力模式实践	dennis1966; miller2003 watson2010	DIRECT STRONG	不可伪造性以熵实现（无硬件标签） “能力层可在通用环境落地”的先例
形式化验证的能力内核（金标准差距）	机器检查证明	klein2009	STRONG	ConsistenCy 无证明——正是 §十九的局限 1
撤销：永久单调 + 经中介	撤销理论	redell1974; miller2003; miller2006	DIRECT	退化（更易）情形；A4/A5 代码可证
vat/事件循环响应式并发 + 访问控制统一	健壮组合	miller2006	DIRECT	与“事件循环内核 + 响应式 Fiber”最同构
LLM 作为不可信提议者	混淆代理人/POLA	hardy1988; miller2003	DIRECT	提示注入 = 现代混淆代理人
能力式 Agent 控制层（平行现代工作）	提示注入的系统性防御	debenedetti2025	STRONG	未经评审预印本；与 CaMeL 论点平行

Consistency 概念	理论	关键文献	强度	备注
intent 门（不可逆效果不内联）	幂等意图/效果分级	debenedetti2025	STRONG	无已确立的“预算化权威”理论（诚实空白）
Fiber 监督生命周期	Actor 语义/OTP 监督树	agha1986; armstrong2003	STRONG	分歧点：单次尝试诚实失败 vs OTP 重启掩盖
ACB 12 状态机 迁移不变 式/fail-closed	带标迁移系统 时序逻辑	keller1976 pnueli1977	DIRECT DIRECT	字面实例 S1-S3 的规范语言
内核为持久响应式运行时	响应式系统	hare1985	STRONG	响应式/变换式二分定位
积 LTS 的自动检验（未接线）	CTL 模型检验	clarke1981	PROPOSED	48 状态积空间是现成对象
工作流 DAG 校验	WF-net 健全性	vanderaalst1998	N/A-for-now	负结果：无环 DAG 上 Kahn 已给终止/无死锁
准入队列词汇/稳定性	Kendall 记号/M/M/c	kendall1953; kleinrock1975	DIRECT	词汇直用；公式是声明近似
积压-逗留不变式	Little 定律	little1961	DIRECT	纪律无关——违背表下仍可用
近容量延迟爆炸 × 变异性	Kingman 重交通	kingman1962	DIRECT	GI/G/1 警示随行
优先级-then-FIFO 饥饿形状	非抢占 HOL	kleinrock1975	DIRECT	P1 截止期取消是兜底而非公平份额
公平份额替代（未实现）	公平排队	demers1989	STRONG	对照框架
预算约束准入（研究模型）	CMDP	altman1999	PROPOSED	现状是可行启发式；预算在授权处执法
LLM 服务时间非指数实证	迭代级调度/连续组批/内存管理	yu2022orca; kwon2023vllm	STRONG	μ 是客户端不可见的隐变量
发现-证据	数据溯源	buneman2001	STRONG	两个溯源问题逐字迁移
why/where 溯源				
prov 四元组结构	PROV-DM	provdms2013	DIRECT	实体/活动/派生词汇的实例化
SHA-256 规范元组身份；快照寻址	内容寻址承诺/哈希树	merkle1989	DIRECT	单叶退化 Merkle

ConsistenCy 概念	理论	关键文献	强度	备注
仅追加审计日志	安全审计日志/防篡改结构	schneier1999; crosby2009	STRONG	对手模型更弱；一致性证明是升级路径
分析器-步骤-材料-产品链	供应链证明	torresarias2019	STRONG	in-toto 同构
模型输出事后接地	事后归因修订	gao2023rarr	STRONG	接地依据是内核证据而非另一模型
原始模型输出不可靠的动机	审查评论处置率实证	goldman2025	STRONG	ASE 2025 实证
神经代码审查基线（任务锚点）	预训练代码审查	li2022codereviewer	STRONG	无接地语义——正是差距
变更触发 vs 周期触发（结构）	事件触发采样/最小事件间隔	astrom1999; tabuada2007	STRONG(结构)	定量最优性 WEAK（无随机被控对象）
单运行去重 + 新到替换	AoI 缓冲-1 洞察	kaul2012	USEFUL	AoI 真正迁移的一条
AoI 词汇/峰值年龄记账	AoI 综述	yates2021	USEFUL	词汇与记账框架
版本滞后 $D(t)$	版本 AoI	buyukates2021	USEFUL + PROPOSED	定义接地； Git-DAG/摘要等价类是扩展
摘要等值触发（无噪声观测）	顺序变化检测	page1954	N/A	负结果：无虚警/延迟权衡可优化
平均检测时延框架	更新报酬	ross1970	USEFUL	时延半边成立；代价半边未建模
Agent 框架对照（无权威中介）	ReAct / AutoGen / MetaGPT	yao2023react; wu2024autogen; hong2024metagpt	对照	工作负载形态而非同类竞争者

表 B.1: 完整文献映射矩阵。强度列的分级记号（DIRECT、STRONG、USEFUL、PROPOSED、WEAK、N/A）与正文六类分级一致。

附录 C 符号表

符号	定义（一行）
$\mathbb{N}, t$	离散时间域；步指标（内核中介循环的串行化序）
X_t	全局状态 $(R_t, S_t, Q_t, \mathfrak{A}_t, C_t, V_t, K_t, E_t, P_t, B_t)$
R_t	外部仓库状态（HEAD、对象库、工作树）——经观测摘要部分可观测
S_t	不可变钉定快照集 $(repo, sha)$
Q_t	准入队列（优先级 + FIFO 序）
$\mathfrak{A}_t$	ACB 注册表：AgentId $\rightarrow$ 12 状态 LTS 状态
C_t	能力表：句柄 $\rightarrow$ 记录 (subject/runId/action/resource/scope/expiresAt/revoked/budgetRef)
V_t	上下文可见性：内容寻址存储 $H : \text{text} \rightarrow \{0, 1\}^{256}$ 上的逐 Agent 页表 PT_a
K_t	共效/Fiber 可用性状态
E_t	证据库（不可变记录，内容派生指纹）
P_t	提供商状态（密钥、可用性、有效速率）——隐藏
B_t	逐能力预算态 $(used_c, reserved_c)$ ，不变式 $used + reserved \leq max$
Π, Π_{intent}	受保护系统调用动作集（12 项）；意图子集 $\{\text{repo.write, github.publish}\}$
$q = (h, a, \alpha, r, s)$	系统调用请求：句柄、主语、动作、资源、作用域
$\kappa(r)$	资源类别函数（ χ_5 使用）
$\chi_1 \cdots \chi_9$	九个授权检查（定义 4.1）
Auth	授权谓词 $\bigwedge_i \chi_i$ （有序首败拒绝）
Exec, $m(q)$, $\prec_{\text{ser}}$	执行事件；其中介点；内核中介循环串行化序
S1, S2, S3	安全不变式（§四、§4.4、命题 8.2）
$t_r, t_{\text{unload}}, \delta_{\text{prop}}$	撤销时刻；Fiber 卸载完成时刻；传播延迟
W_{stale}	陈旧门面窗口 $[t_r, t_{\text{unload}}]$
Available, Visible	共效/Fiber 可用性；工作集页可见性
$S_{\text{ACB}}, \rightarrow_A, T_{\text{term}}$	12 状态 ACB LTS、其迁移关系、吸收终态集
$S_F, \rightarrow_F$	4 状态 Fiber LTS（Cordis 库语义）
$\varphi, \parallel_{\varphi}$	单向调度器 $\rightarrow_{\text{Fiber}}$ 耦合；受限异步积
$G = (V, E), \sigma$	工作流 DAG；确定性拓扑序
$\mathcal{D}_t, Y_t, \varphi(\text{数据谓词})$	数据面坐标；权威坐标；数据面命题（重载 φ 由语境区分）

符号	定义（一行）
$\mathcal{V}$	裁决泛函 $(q, X) \mapsto \text{Auth}(q)$ （命题 8.2）
validate, compile, dryload, execute	workflow 四段管线函数
$e, \text{prov}, N, \text{canonical}, \text{find}$	证据记录；溯源；store 规范化；规范 JSON；SHA-256 指纹
A_v	版本钉定的全函数确定性分析器
ground	确定性接地分类器 $\text{Finding}^* \rightarrow \{\text{accept}, \text{downgrade}, \text{reject}\}^*$
$\lambda, \mu, \tau, c, \rho, A_e$	到达率、服务率、均值服务、并发上限、利用率、offered load (erlangs)
$P_{\text{wait}}, W_q, W, L, L_q$	Erlang-C 等待概率；排队/总逗留；Little 关系
c_a^2, c_s^2	到达间隔/服务时间的平方变异系数 (Kingman)
R_k, W_0	类 $\leq k$ 累积利用率；平均残余工作 (HOL) —— 注意与仓库状态 R_t 区分（下标）
$\mathcal{S}, \mathcal{A}, P, c, g_k, B_k, \gamma, \pi$	CMDP 组元；折扣；策略
$\Delta(t), D(t), d(\cdot, \cdot), \Delta_v(t)$	时间年龄（已拒绝）；版本滞后；DAG 拟距离；版本 AoI
$T_{\text{poll}}, \delta_{\text{scan}}, T_{\text{deb}}, T_{\text{run}}$	轮询周期、扫描时长、防抖（驻留时间）、运行时长； $W = \sum$ （尾随窗口）
θ, λ_c	变更时刻；泊松变更率 (A10)
Φ, u_t, ζ_t	统一模型演化映射；控制；外生噪声 (PROPOSED)
A1–A10	常设假设 (A1–A3 中介、A4–A5 撤销、A6–A8 分析器/规范、A7' 单射、A9–A10 新鲜度)
F1–F5	运行活性的公平/终止假设（命题 8.3）

表 C.1: 符号表。重载符号 (R_k vs R_t 、数据谓词 φ vs 耦合 φ) 已由上下文与下标消歧；本表为准。

参考文献

- [1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *The Eleventh International Conference on Learning Representations (ICLR 2023)*. arXiv:2210.03629. 2023.
- [2] Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation”. In: *First Conference on Language Modeling (COLM 2024)*. arXiv:2308.08155. 2024.
- [3] Sirui Hong et al. “MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework”. In: *The Twelfth International Conference on Learning Representations (ICLR 2024)*. arXiv:2308.00352. 2024.
- [4] Saul Goldman, Hong Yi Lin, Jirat Pasuksmit, et al. “What Types of Code Review Comments Do Developers Most Frequently Resolve?” In: *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE 2025)*. arXiv:2510.05450. 2025, pp. 3760–3765. DOI: [10.1109/ASE63991.2025.00312](https://doi.org/10.1109/ASE63991.2025.00312).
- [5] Norm Hardy. “The Confused Deputy: (or why capabilities might have been invented)”. In: *ACM SIGOPS Operating Systems Review* 22.4 (1988), pp. 36–38. DOI: [10.1145/54289.871709](https://doi.org/10.1145/54289.871709).
- [6] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, et al. *Defeating Prompt Injections by Design*. Preprint; introduces the CaMeL capability-based control layer. 2025. arXiv: [2503.18813](https://arxiv.org/abs/2503.18813).
- [7] Jack B. Dennis and Earl C. Van Horn. “Programming Semantics for Multiprogrammed Computations”. In: *Communications of the ACM* 9.3 (1966), pp. 143–155. DOI: [10.1145/365230.365252](https://doi.org/10.1145/365230.365252).
- [8] Jerome H. Saltzer and Michael D. Schroeder. “The Protection of Information in Computer Systems”. In: *Proceedings of the IEEE* 63.9 (1975), pp. 1278–1308. DOI: [10.1109/PROC.1975.9939](https://doi.org/10.1109/PROC.1975.9939).

- [9] Mark S. Miller, Ka-Ping Yee, and Jonathan Shapiro. *Capability Myths Demolished*. Tech. rep. SRL-2003-02. Systems Research Laboratory, Johns Hopkins University, 2003.
- [10] Mark Samuel Miller. “Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control”. PhD thesis. Johns Hopkins University, 2006.
- [11] James P. Anderson. *Computer Security Technology Planning Study*. Tech. rep. ESD-TR-73-51. Vol. I, AD-758 206. Hanscom Field, Bedford, MA: Deputy for Command and Management Systems, HQ Electronic Systems Division, USAF, 1972.
- [12] D. D. Redell and R. S. Fabry. “Selective Revocation of Capabilities”. In: *Proceedings of the IRIA International Workshop on Protection in Operating Systems*. IRIA/LABORIA; bibliographic record reconstructed from peer citations (Flask, USENIX Security 1999; Levin et al., 1975). Rocquencourt, France, 1974, pp. 197–209.
- [13] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, et al. “seL4: Formal Verification of an OS Kernel”. In: *Proceedings of the 22nd ACM SIGOPS Symposium on Operating Systems Principles (SOSP '09)*. 2009, pp. 207–220. DOI: [10.1145/1629575.1629596](https://doi.org/10.1145/1629575.1629596).
- [14] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. “Capicum: Practical Capabilities for UNIX”. In: *Proceedings of the 19th USENIX Security Symposium*. Washington, DC, 2010.
- [15] Santiago Torres-Arias, Hafsa Afzali, Tayssir Kuppusamy, Reza Curtmola, and Justin Cappos. “in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes”. In: *Proceedings of the 28th USENIX Security Symposium*. 2019.
- [16] Matt Bishop and Michael Dilger. “Checking for Race Conditions in File Accesses”. In: *Computing Systems* 9.2 (1996), pp. 131–152.
- [17] David Harel and Amir Pnueli. “On the Development of Reactive Systems”. In: *Logics and Models of Concurrent Systems*. Ed. by Krzysztof R. Apt. Vol. 13. NATO ASI Series F. Berlin: Springer, 1985, pp. 477–498. DOI: [10.1007/978-3-642-82453-1_17](https://doi.org/10.1007/978-3-642-82453-1_17).
- [18] Gul Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press Series in Artificial Intelligence. Cambridge, MA: MIT Press, 1986.

- [19] Joe Armstrong. “Making Reliable Distributed Systems in the Presence of Software Errors”. PhD thesis. Royal Institute of Technology (KTH), Stockholm, Sweden, 2003.
- [20] Robert M. Keller. “Formal Verification of Parallel Programs”. In: *Communications of the ACM* 19.7 (1976), pp. 371–384. DOI: [10.1145/360248.360251](https://doi.org/10.1145/360248.360251).
- [21] Amir Pnueli. “The Temporal Logic of Programs”. In: *18th Annual Symposium on Foundations of Computer Science (FOCS 1977)*. 1977, pp. 46–57. DOI: [10.1109/SFCS.1977.32](https://doi.org/10.1109/SFCS.1977.32).
- [22] Edmund M. Clarke and E. Allen Emerson. “Design and Synthesis of Synchronization Skeletons Using Branching Time Temporal Logic”. In: *Logics of Programs*. Vol. 131. LNCS. Springer, 1981. DOI: [10.1007/BFb0025774](https://doi.org/10.1007/BFb0025774).
- [23] Wil M. P. van der Aalst. “The Application of Petri Nets to Workflow Management: Methods, Technology, and Tools”. In: *Journal of Circuits, Systems and Computers* 8.1 (1998), pp. 21–66.
- [24] David G. Kendall. “Stochastic Processes Occurring in the Theory of Queues and their Analysis by the Method of the Imbedded Markov Chain”. In: *The Annals of Mathematical Statistics* 24.3 (1953), pp. 338–354. DOI: [10.1214/aoms/1177728975](https://doi.org/10.1214/aoms/1177728975).
- [25] Leonard Kleinrock. *Queueing Systems, Volume 1: Theory*. New York: Wiley-Interscience, 1975.
- [26] John D. C. Little. “A Proof for the Queuing Formula: $L = \lambda W$ ”. In: *Operations Research* 9.3 (1961), pp. 383–387. DOI: [10.1287/opre.9.3.383](https://doi.org/10.1287/opre.9.3.383).
- [27] J. F. C. Kingman. “On Queues in Heavy Traffic”. In: *Journal of the Royal Statistical Society: Series B (Methodological)* 24.2 (1962), pp. 383–392.
- [28] Alan Demers, Srinivasan Keshav, and Scott Shenker. “Analysis and Simulation of a Fair Queueing Algorithm”. In: *Proceedings of the ACM SIGCOMM Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication (SIGCOMM '89)*. 1989, pp. 1–12. DOI: [10.1145/75246.75248](https://doi.org/10.1145/75246.75248).
- [29] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. “Orca: A Distributed Serving System for Transformer-Based Generative Models”. In: *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*. 2022.

- [30] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. “Efficient Memory Management for Large Language Model Serving with PagedAttention”. In: *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23)*. 2023. DOI: [10.1145/3600006.3613165](https://doi.org/10.1145/3600006.3613165).
- [31] Eitan Altman. *Constrained Markov Decision Processes*. Vol. 7. Stochastic Modeling. Boca Raton, FL: Chapman & Hall/CRC, 1999.
- [32] Karl J. Åström and Bo Bernhardsson. “Comparison of Periodic and Event Based Sampling for First-Order Stochastic Systems”. In: *Proceedings of the 14th IFAC World Congress*. Beijing, China, 1999.
- [33] Paulo Tabuada. “Event-Triggered Real-Time Scheduling of Stabilizing Control Tasks”. In: *IEEE Transactions on Automatic Control* 52.9 (2007), pp. 1680–1685. DOI: [10.1109/TAC.2007.904277](https://doi.org/10.1109/TAC.2007.904277).
- [34] E. S. Page. “Continuous Inspection Schemes”. In: *Biometrika* 41.1–2 (1954), pp. 100–115.
- [35] Sanjit Kaul, Roy D. Yates, and Marco Gruteser. “Real-Time Status: How Often Should One Update?” In: *Proceedings of IEEE INFOCOM 2012*. 2012, pp. 2731–2735. DOI: [10.1109/INFCOM.2012.6195689](https://doi.org/10.1109/INFCOM.2012.6195689).
- [36] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. “Why and Where: A Characterization of Data Provenance”. In: *Proceedings of the 8th International Conference on Database Theory (ICDT 2001)*. Vol. 1973. LNCS. Springer, 2001, pp. 316–330. DOI: [10.1007/3-540-44503-X_20](https://doi.org/10.1007/3-540-44503-X_20).
- [37] Luc Moreau and Paolo (eds.) Missier. *PROV-DM: The PROV Data Model*. W3C Recommendation. 30 April 2013. 2013.
- [38] Ralph C. Merkle. “A Certified Digital Signature”. In: *Advances in Cryptology — CRYPTO '89*. Vol. 435. LNCS. Hash-tree structure originated in the author’s 1979 Stanford technical report and 1980 IEEE S&P paper. Springer, 1990, pp. 218–238. DOI: [10.1007/0-387-34805-0_21](https://doi.org/10.1007/0-387-34805-0_21).
- [39] Bruce Schneier and John Kelsey. “Secure Audit Logs to Support Computer Forensics”. In: *ACM Transactions on Information and System Security* 2.2 (1999), pp. 159–176. DOI: [10.1145/317087.317089](https://doi.org/10.1145/317087.317089).
- [40] Scott A. Crosby and Dan S. Wallach. “Efficient Data Structures for Tamper-Evident Logging”. In: *Proceedings of the 18th USENIX Security Symposium*. 2009, pp. 317–334.

- [41] Leo Gao et al. “RARR: Researching and Revising What Language Models Say, Using Language Models”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. arXiv:2210.08726. 2023.
- [42] Roy D. Yates, Yin Sun, D. Richard Brown, Sanjit K. Kaul, Eytan Modiano, and Sennur Ulukus. “Age of Information: An Introduction and Survey”. In: *IEEE Journal on Selected Areas in Communications* 39.5 (2021), pp. 1183–1210. DOI: [10.1109/JSAC.2021.3065072](https://doi.org/10.1109/JSAC.2021.3065072).
- [43] Baturalp Buyukates, Melih Bastopcu, and Sennur Ulukus. *Version Age of Information in Clustered Gossip Networks*. 2021. arXiv: [2109.08669](https://arxiv.org/abs/2109.08669).
- [44] Sheldon M. Ross. *Applied Probability Models with Optimization Applications*. Dover reprint, 1992. San Francisco: Holden-Day, 1970.
- [45] Zhiyu Li, Shuai Lu, Daya Guo, et al. “Automating Code Review Activities by Large-scale Pre-training”. In: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2022)*. arXiv:2203.09095. 2022. DOI: [10.1145/3540250.3549081](https://doi.org/10.1145/3540250.3549081).